

Wrocław University of Science and Technology
Faculty of Information and Communication Technology

Field of study: **Algorithmic Computer Science, Cryptography and Computer Security**

MASTER THESIS

**Differential privacy techniques in data
sketches**

Ksawery Możdżyński

Supervisor

dr inż. Jakub Lemiesz

Keywords: data sketch, differential privacy

WROCLAW 2024

ABSTRACT

This thesis provides an overview of the use of differential privacy in data sketches and adapts the FastExpSketch algorithm to differential privacy. It introduces the topics of differential privacy and data sketches, reviews the current state of research on differential privacy in data sketches, and presents a theoretical and experimental analysis of the adapted FastExpSketch algorithm.

STRESZCZENIE

Niniejsza praca magisterska przedstawia przegląd zastosowań prywatności różnicowej w szkicach danych oraz adaptację algorytmu FastExpSketch do wymagań prywatności różnicowej. Praca stanowi wprowadzenie do tematyki prywatności różnicowej i szkiców danych, omawia aktualny stan badań nad prywatnością różnicową w szkicach danych, a także zawiera analizę teoretyczną i eksperymentalną zaadaptowanego algorytmu FastExpSketch.

CONTENTS

Introduction	3
Problem statement	3
Objectives of the thesis	3
Outline of the thesis	3
1. An introduction to differential privacy	5
2. An introduction to data sketches	9
2.1. General description of data sketches	9
2.2. Introduction to the FM sketch	10
3. Overview of articles on the privacy of data sketches	14
3.1. Challenges in preserving privacy when publishing data sketches	14
3.2. Privacy of users in a distributed system	15
3.2.1. Privacy preservation in the FM sketch	15
3.3. The FM sketch itself preserves differential privacy	16
3.4. Differential privacy of a class of cardinality estimators	17
4. Privacy analysis of the Exponential Sketch	19
4.1. Introduction to the Exponential Sketch	19
4.2. Sensitivity of the Exponential Sketch to new elements of the data stream	20
4.3. Differential privacy of the Exponential Sketch	22
4.4. Proof of differential privacy of the modified Exponential Sketch	23
4.4.1. Proof for one-dimensional vectors	24
4.4.2. Proof for multi-dimensional vectors	28
4.5. Reducing noise with the advanced composition theorem	28
4.6. The unmodified Exponential Sketch is approximately differentially private for a certain class of data streams	29
4.7. Differential privacy of the Fast Exponential Sketch	29
5. Attacks on data privacy	33
5.1. Reconstruction attacks	33
5.1.1. A basic example of a reconstruction attack	33
5.1.2. Reconstruction attack on the 2010 US Census data	33
5.1.3. Reconstruction attack on the Diffix system	34
5.2. Theoretical foundations of reconstruction attacks	34

5.3.	A privacy attack on ExpSketch	35
5.3.1.	Simulation of the attack when the transaction significantly affects the data sketch	37
5.3.2.	Simulation of the attack when the transaction does not significantly affect the data sketch	37
6.	Experimental analysis of the adapted Exponential Sketch	40
6.1.	The artificial element does not significantly affect the data stream	40
6.2.	The artificial element significantly affects the data stream	40
	Summary	43
	Bibliography	44
	List of Figures	45
	List of Algorithms	46
	List of Tables	47
	Appendix	48
A.	Proof of Theorem 4.2	49
B.	Proof of Theorem 4.3	55

INTRODUCTION

A crucial aspect of computer science is the ability to analyze large datasets efficiently. For this purpose, data structures called data sketches are used, which provide approximate answers to queries about large datasets. Data sketches are essential when a dataset is so large that exact algorithms would require significantly more time and hardware resources than are available. At the same time, the results of such an analysis must not compromise the privacy of the individuals whose data is included in the analyzed dataset. A commonly used method to ensure this is differential privacy. This thesis examines FastExpSketch in the context of privacy attacks and explores possible methods of implementing differential privacy in this sketch.

PROBLEM STATEMENT

Although data sketches are widely used due to their efficiency in processing large datasets, they may be vulnerable to privacy attacks. Modifying a sketch so that it ensures a proper level of privacy protection without significantly impairing its efficiency and accuracy is a challenge. This thesis examines the theoretical and practical challenges associated with adapting FastExpSketch to differential privacy.

OBJECTIVES OF THE THESIS

The main objectives of this thesis are:

1. to discuss the key concepts related to differential privacy and data sketches;
2. to analyze the literature on differential privacy in the context of data sketches;
3. to propose a differentially private version of the FastExpSketch algorithm;
4. to simulate a privacy attack on FastExpSketch and analyze the obtained results;
5. to evaluate experimentally the impact of differential privacy on the estimation accuracy of FastExpSketch.

OUTLINE OF THE THESIS

The thesis is organized as follows.

Chapter 1 introduces the key concepts, definitions and theorems of differential privacy. It defines pure and approximate differential privacy and the Laplace mechanism for

achieving differential privacy, and it discusses the theorems that make differential privacy a strong notion of privacy protection.

Chapter 2 explains the concept of data sketches using examples such as MinCount and the FM sketch.

Chapter 3 discusses several scientific articles on differential privacy in data sketches.

Chapter 4 begins with an overview of ExpSketch. Then, its sensitivity to changes of individual elements of the data stream is examined. It is shown that as the number of elements in the data stream grows, the influence of a single element on ExpSketch decreases drastically, which can be perceived as an advantage in terms of privacy protection. A modification of ExpSketch that satisfies pure differential privacy is proposed. It is then proven that, for a subset of data streams that excludes extreme cases in which a single element has a significant impact on the stream, ExpSketch without any modifications satisfies approximate differential privacy. All privacy properties proven in this chapter for ExpSketch also hold for FastExpSketch.

Chapter 5 analyzes the literature on privacy attacks. Subsequently, a model is proposed in which an adversary attempts to attack a system that uses FastExpSketch or DP-FastExpSketch. Based on simulations of this model, the adversary's chances of success are compared depending on whether a differentially private data sketch is used.

Chapter 6 compares experimentally the accuracy of the differentially private version of ExpSketch with the regular version.

1. AN INTRODUCTION TO DIFFERENTIAL PRIVACY

Differential privacy is a technique for safeguarding data privacy that enables statistical analysis of a dataset while minimizing the risk of disclosing information about individuals in that dataset. The goal of this technique is to ensure that the results of the analysis do not allow the identification of specific individuals in the dataset.

Differential privacy enables the analysis of datasets that contain sensitive information, such as medical data. For example, a dataset may include the medical histories of individual patients. An analysis using this technique allows drawing general conclusions about the entire dataset, such as correlations between different diseases among patients, while simultaneously protecting the privacy of individuals.

In practice, differential privacy introduces artificial noise into the analysis, which makes its results more general and harder to link to specific individuals. As a result, even someone who has access to the results of the analysis cannot explicitly link them to specific individuals in the dataset.

Before formally defining differential privacy, we need to establish several auxiliary concepts.

Definition 1.1 (Probability simplex [6, p. 16]). Given a discrete set B , the probability simplex over B , denoted $\Delta(B)$, is defined to be

$$\Delta(B) = \left\{ x \in \mathbb{R}^{|B|} : x_i \geq 0 \text{ for all } i \text{ and } \sum_{i=1}^{|B|} x_i = 1 \right\}.$$

Definition 1.2 (Randomized algorithm [6, p. 16]). A randomized algorithm $\mathcal{M}$ with domain A and discrete range B is associated with a mapping $N : A \rightarrow \Delta(B)$. On input $a \in A$, the algorithm $\mathcal{M}$ outputs $\mathcal{M}(a) = b$ with probability $(N(a))_b$ for each $b \in B$.

The first two definitions concern the probability simplex and randomized algorithms. The probability simplex over a set B is the set $\Delta(B)$ whose elements represent all possible probability distributions over the elements of B . A randomized algorithm $\mathcal{M}$ with domain A and discrete range B is the kind of algorithm for which we will require differential privacy. For an argument $a \in A$, $\mathcal{M}$ returns an element $b \in B$ according to the probability distribution corresponding to a specific point of the set $\Delta(B)$.

Definition 1.3 (Database [6, p. 17]). We will think of databases x as being collections of

records from a universe $\mathcal{X}$. More precisely, $x \in \mathbb{N}^{\mathcal{X}}$, where each entry x_i represents the number of elements of type $i \in \mathcal{X}$ in the database x .

According to this definition, a database is a function $x : \mathcal{X} \rightarrow \mathbb{N}$ which, for an element $i \in \mathcal{X}$, returns the number of its occurrences in the database. Such a function can also be written as $x \in \mathbb{N}^{\mathcal{X}}$. For example, a database x containing basic information about all citizens of Poland can be defined as follows. Let $\mathcal{X}$ be the set of all possible combinations of first names, surnames, addresses and valid PESEL numbers. For an element $p \in \mathcal{X}$, the value $x(p)$ is the number of occurrences of p in the database.

Definition 1.4 (ℓ_1 -distance between databases [6, p. 17]). The ℓ_1 norm of a database x is denoted $\|x\|_1$ and is defined to be

$$\|x\|_1 = \sum_{i=1}^{|\mathcal{X}|} |x_i|.$$

The ℓ_1 distance between two databases x and y is $\|x - y\|_1$.

It is worth noting that if the distance between two databases x and y equals 1, then these databases differ in the count of exactly one element.

Definition 1.5 (Differential privacy [6]). A randomized algorithm $\mathcal{M}$ with domain $\mathbb{N}^{|\mathcal{X}|}$ is (ε, δ) -differentially private if for all $S \subseteq \text{Range}(\mathcal{M})$ and for all $x, y \in \mathbb{N}^{|\mathcal{X}|}$ such that $\|x - y\|_1 \leq 1$:

$$\Pr[\mathcal{M}(x) \in S] \leq e^\varepsilon \Pr[\mathcal{M}(y) \in S] + \delta.$$

Intuitively, the definition of differential privacy can be interpreted as follows. Suppose there is a database $x \in \mathbb{N}^{\mathcal{X}}$ to which we do not have direct access, and the algorithm $\mathcal{M}$ is differentially private. Then, regardless of whether a specific element p is present in the database or not, analyzing the result $\mathcal{M}(x)$ does not allow us to determine with high probability whether p was present in the database.

When $\delta = 0$, we speak of *pure differential privacy*; when $\delta > 0$, we speak of *approximate differential privacy*. Unlike pure differential privacy, approximate differential privacy allows a small risk of a privacy breach, which depends on the value of the parameter δ .

Theorem 1.1 (Post-processing [6, p. 19]). Let $\mathcal{M} : \mathbb{N}^{|\mathcal{X}|} \rightarrow R$ be a randomized algorithm that is (ε, δ) -differentially private. Let $f : R \rightarrow R'$ be an arbitrary randomized mapping. Then $f \circ \mathcal{M} : \mathbb{N}^{|\mathcal{X}|} \rightarrow R'$ is (ε, δ) -differentially private.

This theorem implies that any further processing of the output of a differentially private algorithm is safe and does not disclose more private information about the users.

Theorem 1.2 (Group privacy [6, p. 20]). *Any $(\varepsilon, 0)$ -differentially private mechanism $\mathcal{M}$ is $(k\varepsilon, 0)$ -differentially private for groups of size k . That is, for all $x, y \in \mathbb{N}^{|X|}$ such that $\|x - y\|_1 \leq k$ and all $S \subseteq \text{Range}(\mathcal{M})$:*

$$\Pr[\mathcal{M}(x) \in S] \leq e^{k\varepsilon} \Pr[\mathcal{M}(y) \in S].$$

This theorem implies that differential privacy also protects databases that differ in more than one element, but at the expense of a higher risk of revealing private information.

Theorem 1.3 (Basic composition [6, p. 42]). *Let $\mathcal{M}_1 : \mathbb{N}^{|X|} \rightarrow R_1$ be an ε_1 -differentially private algorithm, and let $\mathcal{M}_2 : \mathbb{N}^{|X|} \rightarrow R_2$ be an ε_2 -differentially private algorithm. Then their combination, defined to be $\mathcal{M}_{1,2} : \mathbb{N}^{|X|} \rightarrow R_1 \times R_2$ by the mapping $\mathcal{M}_{1,2}(x) = (\mathcal{M}_1(x), \mathcal{M}_2(x))$, is $(\varepsilon_1 + \varepsilon_2)$ -differentially private.*

It follows from this theorem that a composition of several differentially private mechanisms is also differentially private, but its level of privacy protection decreases with the number of composed mechanisms and depends on their individual parameters.

Theorem 1.4 (Advanced composition [6, p. 52]). *Given target privacy parameters $0 < \varepsilon' < 1$ and $\delta' > 0$, to ensure $(\varepsilon', k\delta + \delta')$ cumulative privacy loss over k mechanisms, it suffices that each mechanism is (ε, δ) -differentially private, where*

$$\varepsilon = \frac{\varepsilon'}{2\sqrt{2k \ln(1/\delta')}}.$$

It follows from Theorem 1.3 that if there are k mechanisms $\mathcal{M}_i$, each of them ε_i -differentially private, then for their composition to be ε -differentially private, the sum of the ε_i must not exceed ε . This condition is satisfied, for example, when $\varepsilon_i = \frac{\varepsilon}{k}$. Unfortunately, for a large number of mechanisms, each mechanism $\mathcal{M}_i$ then has a very small value of ε_i , which can lead to significant inaccuracies. Theorem 1.4 allows us to use approximate differential privacy in such a way that it suffices for each mechanism $\mathcal{M}_i$ to be ε_i -differentially private with $\varepsilon_i \approx \frac{\varepsilon}{\sqrt{k}}$.

The following definitions introduce the basic method of making an algorithm differentially private.

Definition 1.6 (ℓ_1 -sensitivity [6, p. 31]). *The ℓ_1 -sensitivity of a function $f : \mathbb{N}^{|X|} \rightarrow \mathbb{R}^k$ is*

$$\Delta f = \max_{\substack{x, y \in \mathbb{N}^{|X|} \\ \|x - y\|_1 = 1}} \|f(x) - f(y)\|_1.$$

Definition 1.7 (Laplace distribution [6, p. 31]). The Laplace distribution (centered at 0) with scale b is the distribution with probability density function

$$\text{Lap}(x \mid b) = \frac{1}{2b} \exp\left(-\frac{|x|}{b}\right).$$

We will write $\text{Lap}(b)$ to denote the Laplace distribution with scale b .

Definition 1.8 (Laplace mechanism [6, p. 32]). Given any function $f : \mathbb{N}^{|\mathcal{X}|} \rightarrow \mathbb{R}^k$, the Laplace mechanism is defined as

$$\mathcal{M}_L(x, f(\cdot), \varepsilon) = f(x) + (Y_1, \dots, Y_k),$$

where Y_i are independent random variables drawn from $\text{Lap}(\Delta f / \varepsilon)$.

Theorem 1.5 ([6]). *The Laplace mechanism preserves $(\varepsilon, 0)$ -differential privacy.*

The Laplace mechanism adds noise drawn from the Laplace distribution to the output of an algorithm. It makes any algorithm differentially private, provided that we can determine the sensitivity of that algorithm, as defined above.

2. AN INTRODUCTION TO DATA SKETCHES

2.1. GENERAL DESCRIPTION OF DATA SKETCHES

For very large datasets, some algorithms may become impractical. A data sketch is a compact summary of a dataset that, for specific problems, allows us to obtain approximate solutions. The computational cost of creating a data sketch and of computing estimates from it is much lower than that of traditional exact algorithms. The accuracy of an estimate obtained from a data sketch depends on the amount of memory reserved for the sketch. For data sketches, there are mathematical results that relate the amount of memory reserved for the sketch to the accuracy of the estimation; an example of such an analysis for the HyperLogLog sketch is given in [9]. Data sketches are built by online algorithms, which means that the sketch is continuously updated in real time as new data arrives from the stream (Figure 2.1).

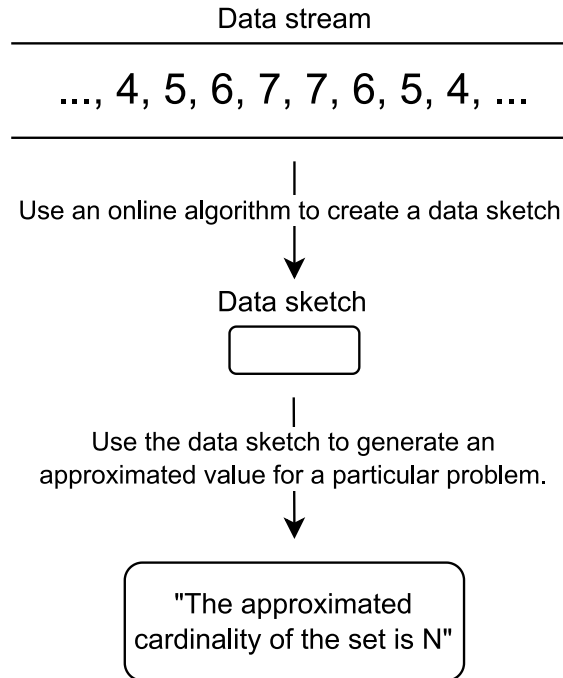


Fig. 2.1: Data sketch usage diagram.

A primary example of the use of data sketches is counting the number of distinct elements in a data stream. The data stream can be defined as a multiset $\mathfrak{M} = (\mathbb{U}, f)$, where $\mathbb{U}$ is a set of unknown size $n = |\mathbb{U}|$, and the function $f : \mathbb{U} \rightarrow \mathbb{N}_+$ gives the multiplicity of

each element of the multiset. An example of a data sketch used to estimate the number of distinct elements in a data stream is MinCount (Algorithm 1).

Algorithm 1 MinCount($\mathfrak{M}, k$)

```

1: Initialization:
2: Set each of the  $k$  positions of the data sketch  $\mathbf{M} = (\mathbf{M}_1, \dots, \mathbf{M}_k)$  to  $\infty$ 
3:
4: Update the data sketch  $\mathbf{M}$  with the elements of the data stream  $\mathfrak{M}$ :
5: for all  $x \in \mathfrak{M}$  do
6:   if  $\mathcal{H}(x) < \mathbf{M}_k$  and  $\mathcal{H}(x) \notin \mathbf{M}$  then                                 $\triangleright$  hash  $\mathcal{H}(x) \in [0, 1)$ 
7:      $\mathbf{M}_k \leftarrow \mathcal{H}(x)$ 
8:     sort( $\mathbf{M}$ )                                                                 $\triangleright$  sort  $\mathbf{M}$  in ascending order
9:   end if
10: end for
11:
12: Estimate the number of distinct elements in the data stream  $\mathfrak{M}$ :
13: if  $\mathbf{M}_k = \infty$  then
14:   return  $|\{i : \mathbf{M}_i \neq \infty\}|$ 
15: else
16:   return  $\frac{k-1}{\mathbf{M}_k}$ 
17: end if

```

Another example of a data sketch is the FM sketch. It is described in more detail below, because Chapter 3 discusses scientific papers based on this sketch. The FM sketch can be useful in the following scenario. Suppose there is a group of users and a set of access points. Any user can connect multiple times to any of the access points. Each access point can use an FM sketch to estimate the number of distinct users who have connected to it.

2.2. INTRODUCTION TO THE FM SKETCH

The operation of the FM sketch is presented in Figure 2.2. Initially, the FM sketch is an array of m fields set to zero. When a user connects to an access point, the user's ID is hashed by a function h_1 to an index of the array, and the corresponding field is set to one. The values of h_1 follow a geometric distribution. When a number of users have connected to the access point and its owner decides to compute an estimate from the sketch, the length L of the uninterrupted sequence of ones starting at the first index of the array is measured. The number of users is estimated as $C = \frac{2^L}{\phi}$, where ϕ is a constant.

Let us assume that the sketch FM_i was created from the elements of the multiset $\mathfrak{M}_i$, and the sketch FM_j was created from the multiset $\mathfrak{M}_j$. The operation $\text{merge}(FM_i, FM_j)$ returns the data sketch that would be created from the elements of the union of the multisets $\mathfrak{M}_i$ and $\mathfrak{M}_j$. If FM_i is the sketch of access point i and FM_j is the sketch of access point j , then the estimate computed from $\text{merge}(FM_i, FM_j)$ approximates the number of users who used access point i or access point j . The merge operation on FM sketches is very

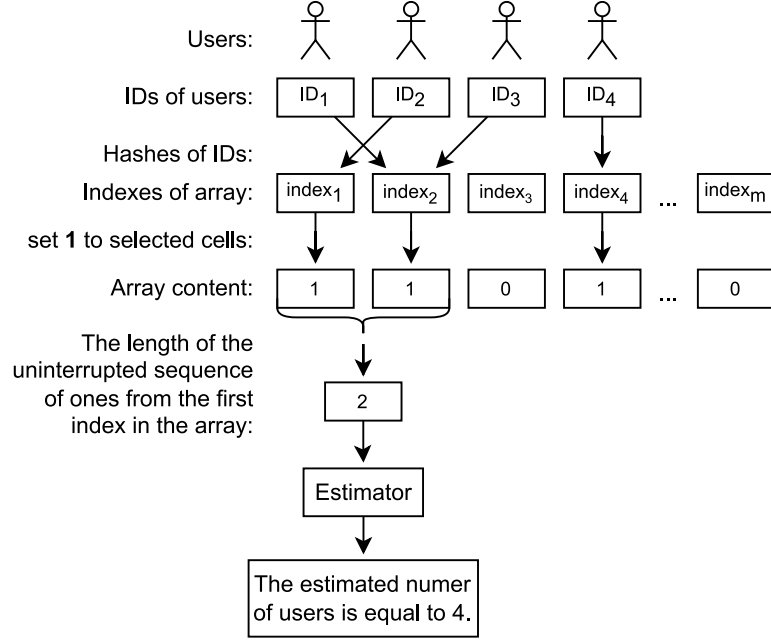


Fig. 2.2: FM sketch.

simple: it suffices to compute the bitwise OR of the corresponding fields of FM_i and FM_j (Figure 2.3).

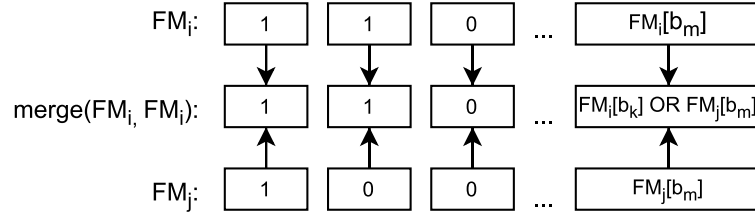


Fig. 2.3: Merge operation of two FM sketches.

The accuracy of the sketch can be improved by combining k instances of the basic FM sketch with an additional hash function h_2 whose values are uniformly distributed over $\{1, \dots, k\}$ (Figure 2.4). When a user connects to an access point that uses the extended FM sketch, one of the k basic FM sketches is first selected by computing $h_2(ID)$. Then, within this sketch, the user's ID is assigned to a field by the function h_1 , as before. Each of the basic sketches stores a sequence of zeros and ones in its array. To compute an estimate, the length L_i of the initial sequence of ones is read from each of the k basic sketches, and the number of users is estimated as

$$C = \frac{k \cdot 2^{\frac{1}{k} \sum_{i=1}^k L_i}}{\phi}.$$

The merge operation on extended sketches FM_i and FM_j performs the basic merge operation on each pair of corresponding components of FM_i and FM_j (Figure 2.5).

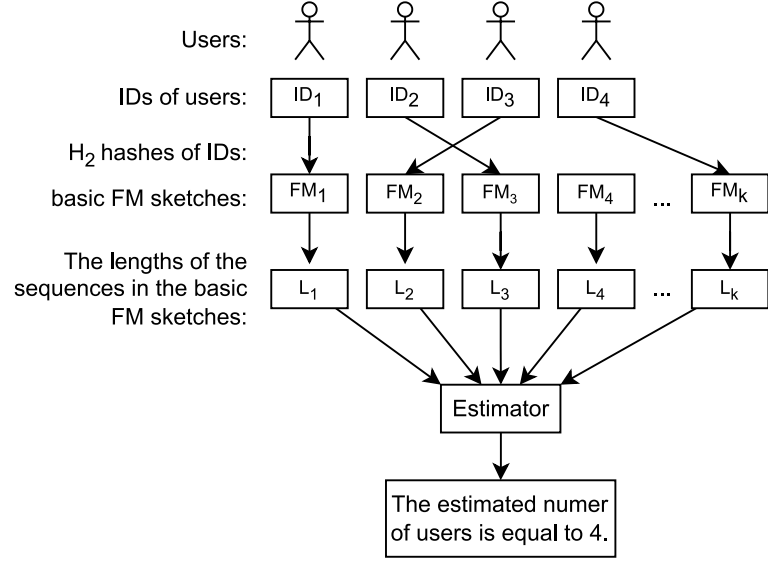


Fig. 2.4: Extended FM sketch.

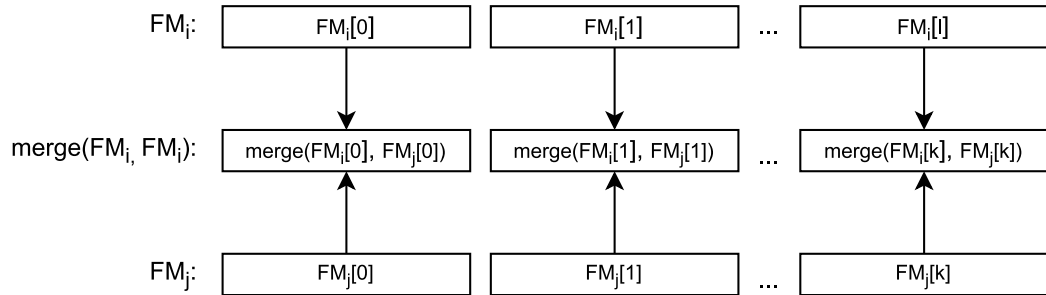


Fig. 2.5: Merge operation of two extended FM sketches.

The extended FM sketch can be interpreted as a $k \times m$ binary matrix in which each row is a basic FM sketch. The merge operation of two such sketches corresponds to the OR operation on the corresponding entries of the matrices:

$$\begin{aligned}\mathbf{FM}_1 &= [a_{ij}]_{(i,j) \in [1,k] \times [1,m]}, & \mathbf{FM}_2 &= [b_{ij}]_{(i,j) \in [1,k] \times [1,m]}, \\ \text{merge}(\mathbf{FM}_1, \mathbf{FM}_2) &= [c_{ij}]_{(i,j) \in [1,k] \times [1,m]},\end{aligned}$$

where $a_{ij}, b_{ij} \in \{0, 1\}$ and $c_{ij} = a_{ij} \vee b_{ij}$.

3. OVERVIEW OF ARTICLES ON THE PRIVACY OF DATA SKETCHES

3.1. CHALLENGES IN PRESERVING PRIVACY WHEN PUBLISHING DATA SKETCHES

The paper [3] analyzes an interesting model of data privacy for sketches (Figure 3.1). Let us assume that a group of users visits various places daily, such as cinemas, theaters or restaurants. Additionally, there is a service that receives a stream of user data: when a user visits a specific place, the information about this visit is added to the data stream. The primary objective of the service is to provide information on the number of people who visited specific places. To achieve this, data sketches designed for cardinality estimation are used. Furthermore, the service should be able to answer more complex queries, such as the number of people who visited both the first and the second place. This requires data sketches that can be merged into one sketch, in such a way that privacy is preserved every time.

The authors proposed a definition of privacy that is weaker than differential privacy. For this definition, they determined a lower bound on the variance of the estimate. More restrictive privacy definitions, such as differential privacy, result in at least the same variance of the estimate.

The main difference between differential privacy and the definition proposed by the authors is the source of the adversary's uncertainty. Under differential privacy, user privacy is preserved through the randomness of the algorithm: even if the adversary knows the data of all users except the target, the target's privacy is still maintained. In the weaker privacy model, it is assumed that the adversary has very limited knowledge of the users' data.

The analysis shows that if a data sketch satisfied the proposed definition of privacy every time a new element of the data stream were added to it, or every time it were merged with another sketch, then the variance of the estimate would grow exponentially with the number of elements. This growth is too large for such a system to be useful in practice. In other words, a data sketch cannot protect privacy after every insertion of a new element, because it would be too inaccurate. To protect users' privacy while maintaining accuracy, one can instead operate on a regular data sketch that does not satisfy privacy requirements

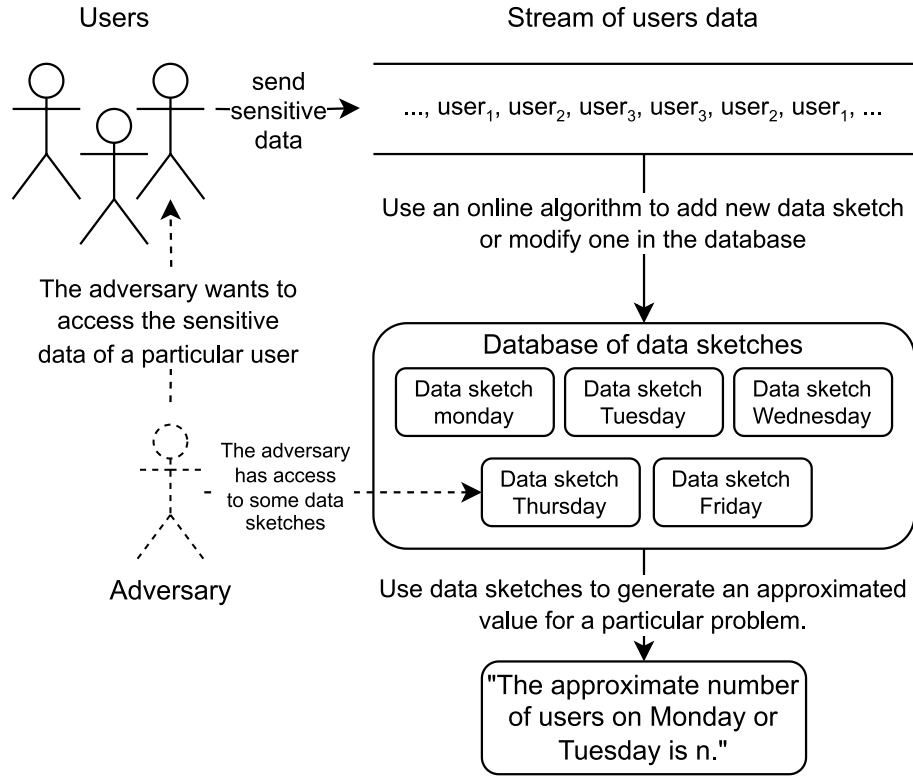


Fig. 3.1: Adversary model from [3].

and introduce privacy protection only when the sketch or an estimate computed from it is to be published.

3.2. PRIVACY OF USERS IN A DISTRIBUTED SYSTEM

Assume that there are N users and M access points, and that each user can connect multiple times to any access point. Each access point uses a data sketch to estimate the number of distinct users who have connected to it. This model is presented in Figure 3.2. The article [15] proposes a method of counting the total number of distinct users across all access points that prevents any access point from gaining sensitive information about the activity of specific users at other access points. The proposed solution uses the FM sketch.

3.2.1. Privacy preservation in the FM sketch

The goal is to calculate the total number of users across all access points in such a way that no access point obtains information about the activity of particular users at other access points. The solution proposed in [15] significantly reduces the leakage of users' private data. The access point responsible for aggregating the data collected from other access points only slightly increases its knowledge about the activity of specific users at the other access points. This increase is small compared to the scenario in which the aggregating access point receives raw data sketches from the other access points.

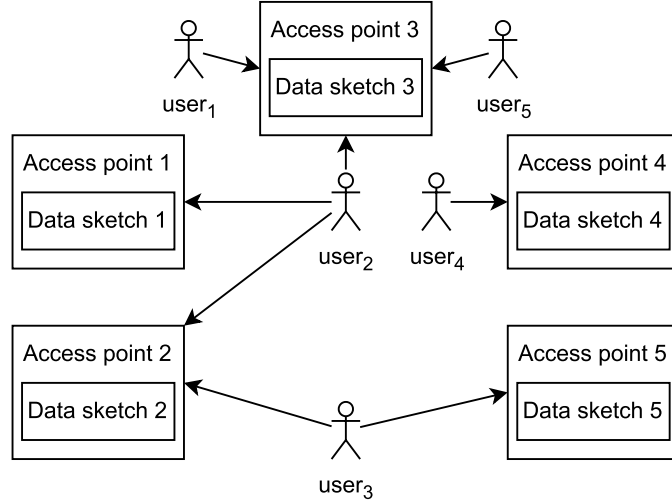


Fig. 3.2: Data sketches in a distributed system, as considered in [15].

Figure 3.3 illustrates how the first access point obtains information about the cumulative activity of users at the other access points. First, the first access point creates a matrix of random integers and sends it to the next access point. This access point checks which fields of its sketch matrix contain ones. If the field of the sketch matrix at coordinates (i, j) equals 1, the access point draws a new random value for the field of the random matrix at coordinates (i, j) . Otherwise, if the field of the sketch matrix at (i, j) equals 0, the value of the field of the random matrix at (i, j) remains unchanged. Once the access point has processed the entire random matrix, it sends it to the next access point. When all access points have processed the random matrix, it returns to the first access point. By comparing its original random matrix with the received one, the first access point can read off a sketch matrix. This matrix is equal to the matrix that would be obtained by merging the sketches of all other access points. The first access point can then merge its own sketch with this matrix and, by computing an estimate from the result, learn the approximate number of distinct users across all access points. At the same time, the first access point cannot determine at which access point the activity of individual users originated. To further enhance user privacy, the access points modify the random matrix not according to their true sketches but according to slightly noised versions of them.

In [15], privacy is not defined as precisely as in [3], and it is not as restrictive as differential privacy. Privacy in this model is preserved mainly under the assumption that there are many access points. With a small number of access points, user privacy would be at risk.

3.3. THE FM SKETCH ITSELF PRESERVES DIFFERENTIAL PRIVACY

The article [14] proves that the FM sketch (in a slightly more general form than the one described above) is (ϵ, δ) -differentially private if the data stream contains at least $\frac{\sqrt{\ln(1/\delta)}}{\epsilon\gamma}$

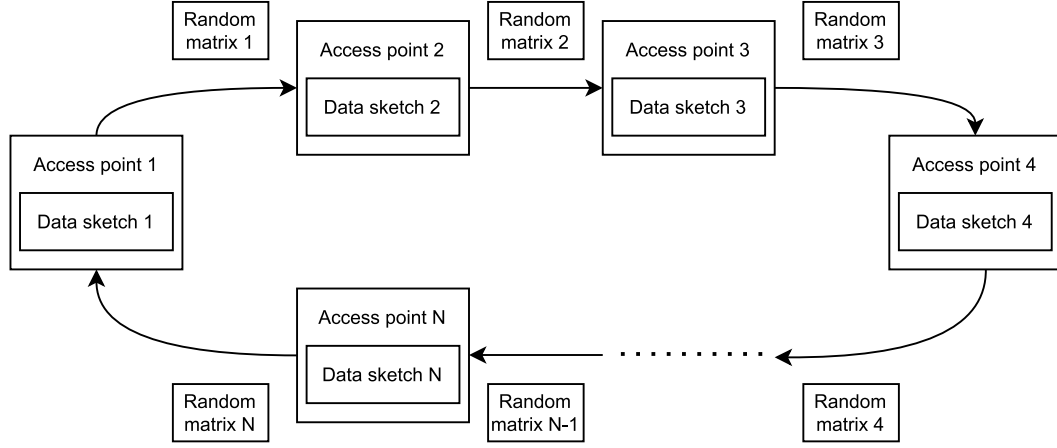


Fig. 3.3: The first access point aggregates data from the other access points.

elements, where γ is one of the parameters of the generalized FM sketch. This property can be used to construct an FM sketch that is differentially private regardless of the number of elements in the data stream: it suffices to add $\frac{\sqrt{\ln(1/\delta)}}{\varepsilon\gamma}$ distinct artificial elements to the data stream and to take this surplus into account when computing the estimate. A similar approach is used in the next chapter to make ExpSketch differentially private.

3.4. DIFFERENTIAL PRIVACY OF A CLASS OF CARDINALITY ESTIMATORS

The previous article dealt with the privacy of the FM sketch. The article [4] discusses the privacy of a whole class of cardinality estimators, namely those that are hash-based, order-invariant and of bounded size. The FM sketch also belongs to this class. The authors proved the following theorem.

Theorem 3.1 ([4]). *A hash-based, order-invariant cardinality estimator of bounded size is ε -differentially private if it satisfies the following two conditions:*

1. *The data stream contains at least $n_0 = \frac{k_{\max}}{1-e^{-\varepsilon}}$ elements, where $k_{\max}$ is a constant determined by the given data sketch.*
2. *Let s denote the internal state of the data sketch and let $\pi(s)$ denote the probability that adding a new element to the sketch in state s changes its state. Then*

$$\sup_s \pi(s) < 1 - e^{-\varepsilon}.$$

In the subsequent part of the article, the authors propose a modification that makes any data sketch from this class differentially private. Let us assume that the data sketch S belongs to this class. The modification is presented in Algorithm 2.

DP-init-Sketch(ε) in line 2 initializes the data sketch S by adding $n_0 = \frac{k_{\max}}{1-e^{-\varepsilon}}$ artificial elements. This ensures that the first condition of Theorem 3.1 is met. Line 7 ensures that each element of the data stream is added to the sketch only with probability π_0 , which

Algorithm 2 DP-cardinality-estimator($\mathfrak{M}, \varepsilon$)

```
1: Initialization:  
2:  $S, n_0 \leftarrow \text{DP-init-Sketch}(\varepsilon)$   
3:  $\pi_0 \leftarrow 1 - e^{-\varepsilon}$   
4:  
5: Update the data sketch  $S$  with the elements of the data stream  $\mathfrak{M}$ :  
6: for all  $x \in \mathfrak{M}$  do  
7:   if  $\mathcal{H}(x) < \pi_0$  then ▷ hash  $\mathcal{H}(x) \in [0, 1)$   
8:      $S.\text{add}(x)$   
9:   end if  
10: end for  
11:  
12: Estimate the number of distinct elements in the data stream  $\mathfrak{M}$ :  
13: return  $\frac{N(S)}{\pi_0} - n_0$ 
```

ensures that the second condition of Theorem 3.1 is met. Since both conditions of the theorem are met, the sketch is differentially private.

4. PRIVACY ANALYSIS OF THE EXPONENTIAL SKETCH

4.1. INTRODUCTION TO THE EXPONENTIAL SKETCH

The ExpSketch data sketch is used to estimate the sum of the attributes of the unique elements of a data stream. The algorithm is presented in Algorithm 3 and is based on the properties of exponential random variables. A detailed description of the algorithm can be found in [12].

Algorithm 3 ExpSketch($\mathfrak{M}, m$)

```

1: Initialization:
2: Set each of the  $m$  positions of the data sketch  $\mathbf{M} = (\mathbf{M}_1, \dots, \mathbf{M}_m)$  to  $\infty$ 
3:
4: Update the data sketch  $\mathbf{M}$  with the elements of the data stream  $\mathfrak{M}$ :
5: for all  $(i, \lambda_i) \in \mathfrak{M}$  do
6:   for all  $j \in \{1, 2, \dots, m\}$  do
7:      $U_{i,j} \leftarrow \mathcal{H}(i \parallel j)$   $\triangleright$  hash  $\mathcal{H}(x) \in [0, 1]$ 
8:      $S_{i,j} \leftarrow -\frac{\ln(U_{i,j})}{\lambda_i}$ 
9:      $\mathbf{M}_j \leftarrow \min\{\mathbf{M}_j, S_{i,j}\}$ 
10:   end for
11: end for
12:
13: Estimate the sum of the attributes of unique elements in  $\mathfrak{M}$ :
14: return  $\frac{m-1}{\sum_{k=1}^m \mathbf{M}_k}$ 

```

In simple terms, the data sketch consists of a vector $\mathbf{M}$ of length m . While the data stream is processed, the values of the fields of $\mathbf{M}$ can only decrease. After all elements of the stream have been processed, the sum of the attributes of the unique elements of the stream is estimated from the vector $\mathbf{M}$.

For each element of the data stream, the algorithm iterates over all fields of the vector $\mathbf{M}$ and attempts to update them. An update consists of generating a value from a particular exponential distribution and assigning it to the corresponding field of $\mathbf{M}$ if it is smaller than the value currently stored in this field. For identical elements of the data stream, the same values are generated. Hence, only the first occurrence of a given element can update the fields of $\mathbf{M}$, and repetitions of elements in the data stream are ignored.

Let us assume that the data sketch has already processed the first k unique elements of the data stream and that a new unique element $(k + 1, \lambda_{k+1})$ arrives. Based on the new element, the algorithm sequentially attempts to update the fields of $\mathbf{M}$. Suppose that the algorithm is updating the j -th field of $\mathbf{M}$. In line 7, a value from the uniform distribution is generated. In line 8, this value is used to generate a value from the exponential distribution $\text{Exp}(\lambda_{k+1})$. At this point, the value stored in the j -th field of $\mathbf{M}$ is a random variable with distribution $\text{Exp}(\sum_{i=1}^k \lambda_i)$, where $\sum_{i=1}^k \lambda_i$ is the sum of the attributes of all unique elements encountered in the stream so far.

After line 9, the j -th field of $\mathbf{M}$ has the distribution $\text{Exp}(\sum_{i=1}^{k+1} \lambda_i)$. This is because for any two independent exponential random variables $X_1 \sim \text{Exp}(S_1)$ and $X_2 \sim \text{Exp}(S_2)$, the random variable $\min(X_1, X_2)$ has the distribution $\text{Exp}(S_1 + S_2)$. After all $k + 1$ elements of the data stream have been processed, the fields of $\mathbf{M}$ store independent values drawn from $\text{Exp}(\sum_{i=1}^{k+1} \lambda_i)$. The expected value of a random variable with this distribution is $(\sum_{i=1}^{k+1} \lambda_i)^{-1}$. Therefore, the expected value of each field of $\mathbf{M}$ is the inverse of the sum of the attributes of the unique elements of the data stream. The average of the fields has the same expected value but a smaller variance. To estimate the sum of the attributes of the unique elements of the stream, it therefore suffices to compute the inverse of the average of the fields of $\mathbf{M}$ (with the bias correction $m - 1$ in place of m).

4.2. SENSITIVITY OF THE EXPONENTIAL SKETCH TO NEW ELEMENTS OF THE DATA STREAM

Assuming that the data sketch $\mathbf{M}$ has already processed some elements of the data stream, this section analyzes the impact of a new element of the stream on the values stored in $\mathbf{M}$. We begin with a single field of $\mathbf{M}$. Let us assume that the data sketch has processed k unique elements. The field $\mathbf{M}_j$ stores a value with distribution $\min(\text{Exp}(\lambda_1), \dots, \text{Exp}(\lambda_k))$, which equals $\text{Exp}(\sum_{i=1}^k \lambda_i)$. Adding a new unique element causes the field to store a value with distribution $\min(\text{Exp}(\lambda_1), \dots, \text{Exp}(\lambda_{k+1}))$. The probability that the field changes its value equals the probability that $X_1 \geq X_2$ for $X_1 \sim \text{Exp}(\sum_{i=1}^k \lambda_i)$ and $X_2 \sim \text{Exp}(\lambda_{k+1})$:

$$P[X_1 \geq X_2] = \frac{\lambda_{k+1}}{\sum_{i=1}^{k+1} \lambda_i}.$$

The field $\mathbf{M}_j$ has the distribution $\text{Exp}(\sum_{i=1}^k \lambda_i)$, and the new candidate value $\mathbf{M}'_j$ that may overwrite it has the distribution $\text{Exp}(\lambda_{k+1})$. The probability that at least one of the m

fields of $\mathbf{M}$ changes is

$$\begin{aligned}
P[\mathbf{M}_1 \geq \mathbf{M}'_1 \vee \dots \vee \mathbf{M}_m \geq \mathbf{M}'_m] &= 1 - \prod_{j=1}^m P[\mathbf{M}_j < \mathbf{M}'_j] \\
&= 1 - P[\mathbf{M}_1 < \mathbf{M}'_1]^m \\
&= 1 - \left(\frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^{k+1} \lambda_i} \right)^m.
\end{aligned}$$

The above refers to the probability that the fields of $\mathbf{M}$ change by any amount. In the context of differential privacy, users' privacy would be compromised if changing a single element of the data stream significantly altered the values of the fields of $\mathbf{M}$.

Let us now calculate the probability that changing one element of the data stream alters the value of a single field of the data sketch by at least T . First, let us examine the random variable Y defined as the difference of two independent random variables $X_1 \sim \text{Exp}(\lambda_A)$ and $X_2 \sim \text{Exp}(\lambda_B)$:

$$Y = X_1 - X_2.$$

The probability density function of the exponential distribution is

$$f_\lambda(x) = \begin{cases} \lambda e^{-\lambda x} & \text{for } x \geq 0, \\ 0 & \text{for } x < 0, \end{cases}$$

and the joint probability density function of X_1 and X_2 is

$$f_{\lambda_A \lambda_B}(x_1, x_2) = \begin{cases} \lambda_A \lambda_B e^{-\lambda_A x_1} e^{-\lambda_B x_2} & \text{for } x_1 \geq 0 \wedge x_2 \geq 0, \\ 0 & \text{otherwise.} \end{cases}$$

For $T \geq 0$ we obtain

$$\begin{aligned}
P(Y \in (-\infty, -T)) &= P(X_1 - X_2 < -T) = P(X_2 > X_1 + T) \\
&= \int_0^\infty \int_{x_1+T}^\infty f_{\lambda_A \lambda_B}(x_1, x_2) \, dx_2 \, dx_1 \\
&= \int_0^\infty \int_{x_1+T}^\infty \lambda_A \lambda_B e^{-\lambda_A x_1} e^{-\lambda_B x_2} \, dx_2 \, dx_1 \\
&= e^{-\lambda_B T} \frac{\lambda_A}{\lambda_A + \lambda_B},
\end{aligned}$$

and

$$\begin{aligned}
P(Y \in (T, \infty)) &= P(X_1 - X_2 > T) = P(X_1 > X_2 + T) \\
&= \int_T^\infty \int_0^{x_1-T} f_{\lambda_A \lambda_B}(x_1, x_2) dx_2 dx_1 \\
&= \int_T^\infty \int_0^{x_1-T} \lambda_A \lambda_B e^{-\lambda_A x_1} e^{-\lambda_B x_2} dx_2 dx_1 \\
&= e^{-\lambda_A T} \frac{\lambda_B}{\lambda_A + \lambda_B}.
\end{aligned}$$

Let us assume that we have two data streams that differ only in the i -th element. We may also assume that the streams contain only unique elements, since repeated elements are ignored. The first stream is

$$\lambda_1, \dots, \lambda_{i-1}, \lambda_i, \lambda_{i+1}, \dots, \lambda_n$$

and the second stream is

$$\lambda_1, \dots, \lambda_{i-1}, \lambda'_i, \lambda_{i+1}, \dots, \lambda_n.$$

The sum of the attributes of the first stream is $\Lambda = \sum_{i=1}^n \lambda_i$, and the sum of the attributes of the second stream is $\Lambda' = \Lambda - \lambda_i + \lambda'_i = \Lambda + \Delta$. The first stream corresponds to the random variable $X_1 \sim \text{Exp}(\Lambda)$, and the second to the random variable $X_2 \sim \text{Exp}(\Lambda')$. Hence

$$\begin{aligned}
P(|X_1 - X_2| \geq T) &= P(X_1 - X_2 \in (-\infty, -T]) + P(X_1 - X_2 \in [T, \infty)) \\
&= e^{-\Lambda' T} \frac{\Lambda}{\Lambda + \Lambda'} + e^{-\Lambda T} \frac{\Lambda'}{\Lambda + \Lambda'}.
\end{aligned}$$

If the i -th element has only a small impact on the sum of the attributes of the streams ($\Lambda \approx \Lambda'$), then the probability that the distance between $\mathbf{M}_j$ and $\mathbf{M}'_j$ exceeds T can be approximated by $e^{-\Lambda T}$.

The standard method of making an algorithm f differentially private is to add Laplace noise $\text{Lap}(\Delta f / \epsilon)$, where Δf is the sensitivity of f . According to Definition 1.6, the sensitivity of f is the maximum distance between the outputs of f for two neighboring datasets. This method cannot be applied to ExpSketch, because $\Delta f = \infty$, so a different approach is necessary. Fortunately, as shown above, large changes for neighboring datasets are extremely rare: a single element causes a noticeable change of the output only if it significantly changes the total sum of the attributes of the unique elements in the dataset.

4.3. DIFFERENTIAL PRIVACY OF THE EXPONENTIAL SKETCH

Unfortunately, the method from [4] cannot be applied directly to ExpSketch, because ExpSketch is not a cardinality estimator and is not strictly hash-based. The hash-based

property is defined in such a way that the data sketch never uses the elements of the data stream directly, but only their hashes. This condition is satisfied, for example, by the MinCount sketch, where the elements are used only to compute their hashes. ExpSketch, in contrast, uses both the hashes and the attributes of the elements. The hash-based property is used many times in the proof of the method, so generalizing it to sketches that are not hash-based seems very challenging.

Interestingly, the modification of ExpSketch presented in Algorithm 4 resembles the method based on Theorem 3.1. Instead of adding $\frac{k_{\max}}{1-e^{-\varepsilon}}$ unique elements to the data sketch, as required by the first condition of that theorem, the modified ExpSketch adds a single large element with attribute $\frac{\lambda_{\max}}{e^{\varepsilon}-1}$. Alternatively, the modification could add $\frac{\lambda_{\max}}{e^{\varepsilon}-1}$ unique elements with attributes equal to 1.

This similarity may come from the fact that if the data stream processed by ExpSketch consists of unique elements (in terms of identifiers) whose attributes are all equal to 1, then ExpSketch can reasonably be regarded as a hash-based cardinality estimator. In that case, ExpSketch would belong to the class of algorithms covered by Theorem 3.1.

Algorithm 4 is our proposal for implementing differential privacy in ExpSketch. First, the data stream is modified. Subsequently, the basic ExpSketch algorithm builds a data sketch from the modified data stream. Finally, the resulting data sketch is also slightly modified. Thus, differential privacy is ensured by modifying both the input and the output of ExpSketch. The algorithm requires two additional parameters: $\lambda_{\max}$ and ε .

In line 2, the data stream is modified: attributes exceeding the limit $\lambda_{\max}$ are reduced to this limit, and a new element with a unique identifier and a very large attribute is added. Subsequently, in line 5, the basic version of ExpSketch is run on the modified data stream. The result is the vector $\mathbf{M}$, which can be used to estimate the sum of the attributes of unique elements; however, an estimate computed from this vector would not yet be differentially private. In line 9, each field of $\mathbf{M}$ that exceeds the limit $\frac{\varepsilon}{\lambda_{\max}}$ is reduced to this limit. This does not significantly affect the estimate if the sum of the attributes of unique elements in the original data stream is much greater than $\frac{\lambda_{\max}}{\varepsilon}$. From this point on, the modified vector $\mathbf{M}'$ is differentially private, and so is any further processing of it. In line 13, an estimate is computed from $\mathbf{M}'$ and reduced by the attribute of the artificial element added in line 2.

Theorem 4.1. *DP-ExpSketch is $(m\varepsilon, 0)$ -differentially private, where m is the size of the sketch.*

4.4. PROOF OF DIFFERENTIAL PRIVACY OF THE MODIFIED EXPONENTIAL SKETCH

Let us assume that there are two data streams $\mathfrak{M}_1$ and $\mathfrak{M}_2$ differing in one element: the stream $\mathfrak{M}_1$ contains the element (i, λ_i) , and the stream $\mathfrak{M}_2$ contains the element (i, λ'_i) . Without loss of generality, we can assume that the streams contain only unique

Algorithm 4 DP-ExpSketch($\mathfrak{M}, m, \lambda_{\text{MAX}}, \varepsilon$)

```
1: Modify  $\mathfrak{M}$  (a preliminary step ensuring differential privacy):
2:  $\mathfrak{M}' \leftarrow \left\{ \left( \text{unique identifier}, \frac{\lambda_{\text{MAX}}}{e^{\varepsilon}-1} \right) \right\} \cup \left\{ (i, \min(\lambda_i, \lambda_{\text{MAX}})) : (i, \lambda_i) \in \mathfrak{M} \right\}$ 
3:
4: Build the data sketch from the modified data stream  $\mathfrak{M}'$ :
5:  $\mathbf{M} \leftarrow \text{ExpSketch}(\mathfrak{M}', m)$ 
6:
7: After the following modification, the privacy conditions are satisfied:
8: for all  $i \in \{1, 2, \dots, m\}$  do
9:    $\mathbf{M}'_i \leftarrow \min \left( \mathbf{M}_i, \frac{\varepsilon}{\lambda_{\text{MAX}}} \right)$ 
10: end for
11:
12: Estimate the sum of the attributes of unique elements in  $\mathfrak{M}$ :
13: return  $\frac{m-1}{\sum_{k=1}^m \mathbf{M}'_k} - \frac{\lambda_{\text{MAX}}}{e^{\varepsilon}-1}$ 
```

elements, since repeated elements are ignored. The possible outputs of DP-ExpSketch for both streams are vectors of m positive real numbers. For brevity, let $\mathcal{A}_m(\mathfrak{M})$ denote $\text{DP-ExpSketch}(\mathfrak{M}, m, \lambda_{\text{MAX}}, \varepsilon)$. Let S be a subset of the possible outputs of DP-ExpSketch. We would like to prove that

$$P(\mathcal{A}_m(\mathfrak{M}_1) \in S) \leq e^{m\varepsilon} P(\mathcal{A}_m(\mathfrak{M}_2) \in S).$$

4.4.1. Proof for one-dimensional vectors

Let us first prove that DP-ExpSketch is differentially private for $m = 1$, that is, when it returns one-dimensional vectors. Let Λ denote the sum of the attributes of the unique elements of the modified data stream $\mathfrak{M}_1$, and let X_1 be a random variable with distribution $\text{Exp}(\Lambda)$. Similarly, let Λ' denote the sum of the attributes of the unique elements of the modified data stream $\mathfrak{M}_2$, and let X_2 be a random variable with distribution $\text{Exp}(\Lambda')$. Then $\mathcal{A}_1(\mathfrak{M}_1)$ and $\mathcal{A}_1(\mathfrak{M}_2)$ correspond to the random variables $\min \left(X_1, \frac{\varepsilon}{\lambda_{\text{MAX}}} \right)$ and $\min \left(X_2, \frac{\varepsilon}{\lambda_{\text{MAX}}} \right)$, respectively. The probability $P(\mathcal{A}_1(\mathfrak{M}_1) \in S)$ depends on whether $\frac{\varepsilon}{\lambda_{\text{MAX}}}$ belongs to S . Let us consider two cases.

1. Assume that $\frac{\varepsilon}{\lambda_{\text{MAX}}} \notin S$. We first prove that

$$\min \left(X_1, \frac{\varepsilon}{\lambda_{\text{MAX}}} \right) \in S \iff X_1 \in S \cap \left(0, \frac{\varepsilon}{\lambda_{\text{MAX}}} \right). \quad (4.1)$$

- (a) Assume that the left-hand side of (4.1) is true, i.e. $\min \left(X_1, \frac{\varepsilon}{\lambda_{\text{MAX}}} \right) \in S$. Then $X_1 < \frac{\varepsilon}{\lambda_{\text{MAX}}}$, because otherwise

$$\min \left(X_1, \frac{\varepsilon}{\lambda_{\text{MAX}}} \right) = \frac{\varepsilon}{\lambda_{\text{MAX}}} \in S,$$

which contradicts the assumption that $\frac{\varepsilon}{\lambda_{\text{MAX}}} \notin S$. Consequently,

$$\min\left(X_1, \frac{\varepsilon}{\lambda_{\text{MAX}}}\right) = X_1 \in S \quad \text{and} \quad X_1 < \frac{\varepsilon}{\lambda_{\text{MAX}}},$$

hence $X_1 \in S \cap \left(0, \frac{\varepsilon}{\lambda_{\text{MAX}}}\right)$. This proves the implication

$$\min\left(X_1, \frac{\varepsilon}{\lambda_{\text{MAX}}}\right) \in S \implies X_1 \in S \cap \left(0, \frac{\varepsilon}{\lambda_{\text{MAX}}}\right). \quad (4.2)$$

(b) Assume that the right-hand side of (4.1) is true, i.e. $X_1 \in S \cap \left(0, \frac{\varepsilon}{\lambda_{\text{MAX}}}\right)$. Then $X_1 < \frac{\varepsilon}{\lambda_{\text{MAX}}}$ and $X_1 \in S$, so

$$\min\left(X_1, \frac{\varepsilon}{\lambda_{\text{MAX}}}\right) = X_1 \in S.$$

This proves the implication

$$\min\left(X_1, \frac{\varepsilon}{\lambda_{\text{MAX}}}\right) \in S \iff X_1 \in S \cap \left(0, \frac{\varepsilon}{\lambda_{\text{MAX}}}\right). \quad (4.3)$$

From the implications (4.2) and (4.3), the equivalence (4.1) holds. It follows that

$$P\left(\min\left(X_1, \frac{\varepsilon}{\lambda_{\text{MAX}}}\right) \in S\right) = P\left(X_1 \in S \cap \left(0, \frac{\varepsilon}{\lambda_{\text{MAX}}}\right)\right). \quad (4.4)$$

Since X_1 has the exponential distribution $\text{Exp}(\Lambda)$, the probability (4.4) is equal to

$$P\left(\min\left(X_1, \frac{\varepsilon}{\lambda_{\text{MAX}}}\right) \in S\right) = \int_{S \cap \left(0, \frac{\varepsilon}{\lambda_{\text{MAX}}}\right)} \Lambda e^{-\Lambda x} dx. \quad (4.5)$$

2. Assume now that $\frac{\varepsilon}{\lambda_{\text{MAX}}} \in S$. Then

$$\begin{aligned} P\left(\min\left(X_1, \frac{\varepsilon}{\lambda_{\text{MAX}}}\right) \in S\right) &= P\left(\min\left(X_1, \frac{\varepsilon}{\lambda_{\text{MAX}}}\right) \in S \setminus \left\{\frac{\varepsilon}{\lambda_{\text{MAX}}}\right\}\right) \\ &\quad + P\left(\min\left(X_1, \frac{\varepsilon}{\lambda_{\text{MAX}}}\right) = \frac{\varepsilon}{\lambda_{\text{MAX}}}\right). \end{aligned} \quad (4.6)$$

By the previous case, the first term of the sum (4.6) is equal to

$$P\left(\min\left(X_1, \frac{\varepsilon}{\lambda_{\text{MAX}}}\right) \in S \setminus \left\{\frac{\varepsilon}{\lambda_{\text{MAX}}}\right\}\right) = \int_{S \cap \left(0, \frac{\varepsilon}{\lambda_{\text{MAX}}}\right)} \Lambda e^{-\Lambda x} dx. \quad (4.7)$$

Using the cumulative distribution function of the exponential distribution, the second term of the sum (4.6) is equal to

$$\begin{aligned} P\left(\min\left(X_1, \frac{\varepsilon}{\lambda_{\text{MAX}}}\right) = \frac{\varepsilon}{\lambda_{\text{MAX}}}\right) &= P\left(X_1 \geq \frac{\varepsilon}{\lambda_{\text{MAX}}}\right) \\ &= 1 - P\left(X_1 < \frac{\varepsilon}{\lambda_{\text{MAX}}}\right) = e^{-\Lambda \frac{\varepsilon}{\lambda_{\text{MAX}}}}. \end{aligned} \quad (4.8)$$

From (4.6), (4.7) and (4.8) we obtain

$$P\left(\min\left(X_1, \frac{\varepsilon}{\lambda_{\text{MAX}}}\right) \in S\right) = \int_{S \cap (0, \frac{\varepsilon}{\lambda_{\text{MAX}}})} \Lambda e^{-\Lambda x} dx + e^{-\Lambda \frac{\varepsilon}{\lambda_{\text{MAX}}}}. \quad (4.9)$$

Based on (4.5) and (4.9), the probability that the output of DP-ExpSketch belongs to S can be expressed as

$$P(\mathcal{A}_1(\mathfrak{M}_1) \in S) = \begin{cases} \int_{S \cap (0, \frac{\varepsilon}{\lambda_{\text{MAX}}})} \Lambda e^{-\Lambda x} dx + e^{-\Lambda \frac{\varepsilon}{\lambda_{\text{MAX}}}} & \text{if } \frac{\varepsilon}{\lambda_{\text{MAX}}} \in S, \\ \int_{S \cap (0, \frac{\varepsilon}{\lambda_{\text{MAX}}})} \Lambda e^{-\Lambda x} dx & \text{if } \frac{\varepsilon}{\lambda_{\text{MAX}}} \notin S, \end{cases} \quad (4.10)$$

and analogously

$$P(\mathcal{A}_1(\mathfrak{M}_2) \in S) = \begin{cases} \int_{S \cap (0, \frac{\varepsilon}{\lambda_{\text{MAX}}})} \Lambda' e^{-\Lambda' x} dx + e^{-\Lambda' \frac{\varepsilon}{\lambda_{\text{MAX}}}} & \text{if } \frac{\varepsilon}{\lambda_{\text{MAX}}} \in S, \\ \int_{S \cap (0, \frac{\varepsilon}{\lambda_{\text{MAX}}})} \Lambda' e^{-\Lambda' x} dx & \text{if } \frac{\varepsilon}{\lambda_{\text{MAX}}} \notin S. \end{cases} \quad (4.11)$$

To complete the proof of differential privacy in the one-dimensional case, we still need an additional inequality, which follows from the modifications introduced in DP-ExpSketch.

4.4.1.1. Inequality resulting from the modification of the Exponential Sketch

In line 2, the data streams are modified: the attributes of elements exceeding the limit λ_{MAX} are reduced to this limit, and an element with a very large attribute $\frac{\lambda_{\text{MAX}}}{e^\varepsilon - 1}$ is added to both $\mathfrak{M}_1$ and $\mathfrak{M}_2$. These modifications lead to the following important conclusions.

1. Let $\Delta = \Lambda - \Lambda'$ be the difference between the sums of the attributes of unique elements in the modified streams $\mathfrak{M}_1$ and $\mathfrak{M}_2$. Since the modified streams differ in only one element, $\Delta = \min(\lambda_i, \lambda_{\text{MAX}}) - \min(\lambda'_i, \lambda_{\text{MAX}})$. Due to the limitation of the attributes in the modified data streams, $-\lambda_{\text{MAX}} \leq \Delta \leq \lambda_{\text{MAX}}$.
2. Due to the added element with attribute $\frac{\lambda_{\text{MAX}}}{e^\varepsilon - 1}$, the sum of the attributes of unique elements in the modified stream $\mathfrak{M}_1$ is not less than $\frac{\lambda_{\text{MAX}}}{e^\varepsilon - 1}$; the same holds for $\mathfrak{M}_2$. Thus $\Lambda \geq \frac{\lambda_{\text{MAX}}}{e^\varepsilon - 1}$ and $\Lambda' \geq \frac{\lambda_{\text{MAX}}}{e^\varepsilon - 1}$, which implies $1 + \frac{\lambda_{\text{MAX}}}{\Lambda'} \leq e^\varepsilon$ and $1 + \frac{\lambda_{\text{MAX}}}{\Lambda} \leq e^\varepsilon$.

3. The above conclusions can be used to prove that for $x \in S \cap \left(0, \frac{\varepsilon}{\lambda_{\text{MAX}}}\right)$ the inequality $e^{\ln(\frac{\Lambda}{\Lambda'}) - x(\Lambda - \Lambda')} \leq e^\varepsilon$ holds. This inequality will be used later in the proof. Let us consider two cases, depending on whether $\Lambda > \Lambda'$.

- (a) If $\Lambda \leq \Lambda'$, then $\ln\left(\frac{\Lambda}{\Lambda'}\right) \leq 0$. It follows that $e^{\ln(\frac{\Lambda}{\Lambda'}) - x(\Lambda - \Lambda')} \leq e^{-x(\Lambda - \Lambda')} = e^{x(\Lambda' - \Lambda)}$. By conclusion 1., $\Lambda' - \Lambda = -\Delta \leq \lambda_{\text{MAX}}$. Since the function $t \mapsto e^t$ is increasing, $e^{x(\Lambda' - \Lambda)} \leq e^{x\lambda_{\text{MAX}}}$. We assumed that $x \in S \cap \left(0, \frac{\varepsilon}{\lambda_{\text{MAX}}}\right)$, so $x \leq \frac{\varepsilon}{\lambda_{\text{MAX}}}$. Using the monotonicity of $t \mapsto e^t$ again, we obtain $e^{x\lambda_{\text{MAX}}} \leq e^{\frac{\varepsilon}{\lambda_{\text{MAX}}} \lambda_{\text{MAX}}} = e^\varepsilon$.
- (b) If $\Lambda > \Lambda'$, then, since $x \in S \cap \left(0, \frac{\varepsilon}{\lambda_{\text{MAX}}}\right)$, we have $x > 0$ and $x(\Lambda - \Lambda') > 0$. By the monotonicity of $t \mapsto e^t$, $e^{\ln(\frac{\Lambda}{\Lambda'}) - x(\Lambda - \Lambda')} \leq e^{\ln(\frac{\Lambda}{\Lambda'})} = \frac{\Lambda}{\Lambda'}$. Since $\Delta = \Lambda - \Lambda'$, we have $\frac{\Lambda}{\Lambda'} = \frac{\Lambda' + \Delta}{\Lambda'} = 1 + \frac{\Delta}{\Lambda'}$. By conclusion 1., $\Delta \leq \lambda_{\text{MAX}}$, so $1 + \frac{\Delta}{\Lambda'} \leq 1 + \frac{\lambda_{\text{MAX}}}{\Lambda'}$. By conclusion 2., $1 + \frac{\lambda_{\text{MAX}}}{\Lambda'} \leq e^\varepsilon$. Hence $e^{\ln(\frac{\Lambda}{\Lambda'}) - x(\Lambda - \Lambda')} \leq 1 + \frac{\lambda_{\text{MAX}}}{\Lambda'} \leq e^\varepsilon$.

We have thus proved the inequality

$$e^{\ln(\frac{\Lambda}{\Lambda'}) - x(\Lambda - \Lambda')} \leq e^\varepsilon \quad \text{for } x \in S \cap \left(0, \frac{\varepsilon}{\lambda_{\text{MAX}}}\right), \quad (4.12)$$

which will be used in the remaining part of the proof.

4.4.1.2. Completing the proof for one-dimensional vectors

Let $x \in S \cap \left(0, \frac{\varepsilon}{\lambda_{\text{MAX}}}\right)$. The left-hand side of inequality (4.12) is equal to $\frac{\Lambda e^{-x\Lambda}}{\Lambda' e^{-x\Lambda'}}$, so the inequality $\frac{\Lambda e^{-x\Lambda}}{\Lambda' e^{-x\Lambda'}} \leq e^\varepsilon$ also holds. It can be easily transformed into $e^\varepsilon \Lambda' e^{-x\Lambda'} - \Lambda e^{-x\Lambda} \geq 0$. Since this inequality holds for every $x \in S \cap \left(0, \frac{\varepsilon}{\lambda_{\text{MAX}}}\right)$,

$$\int_{S \cap \left(0, \frac{\varepsilon}{\lambda_{\text{MAX}}}\right)} \left(e^\varepsilon \Lambda' e^{-x\Lambda'} - \Lambda e^{-x\Lambda} \right) dx \geq 0.$$

By the linearity of the integral,

$$\int_{S \cap \left(0, \frac{\varepsilon}{\lambda_{\text{MAX}}}\right)} \Lambda e^{-x\Lambda} dx \leq e^\varepsilon \int_{S \cap \left(0, \frac{\varepsilon}{\lambda_{\text{MAX}}}\right)} \Lambda' e^{-x\Lambda'} dx. \quad (4.13)$$

By (4.10) and (4.11), the probability that the output of DP-ExpSketch belongs to S depends on whether $\frac{\varepsilon}{\lambda_{\text{MAX}}} \in S$. Let us consider two cases.

1. Assume that $\frac{\varepsilon}{\lambda_{\text{MAX}}} \notin S$. Applying (4.10) and (4.11) to inequality (4.13), we obtain

$$P(\mathcal{A}_1(\mathfrak{M}_1) \in S) \leq e^\varepsilon P(\mathcal{A}_1(\mathfrak{M}_2) \in S).$$

2. Assume that $\frac{\varepsilon}{\lambda_{\text{MAX}}} \in S$. Let us start by proving that

$$e^{-\Lambda \frac{\varepsilon}{\lambda_{\text{MAX}}}} \leq e^\varepsilon e^{-\Lambda' \frac{\varepsilon}{\lambda_{\text{MAX}}}}. \quad (4.14)$$

This inequality is equivalent to $\frac{e^{-\Lambda \frac{\varepsilon}{\lambda_{\text{MAX}}}}}{e^{-\Lambda' \frac{\varepsilon}{\lambda_{\text{MAX}}}}} \leq e^\varepsilon$, that is, to $e^{\frac{\varepsilon}{\lambda_{\text{MAX}}}(\Lambda' - \Lambda)} \leq e^\varepsilon$. It holds because, as shown at the beginning of Section 4.4.1.1, $\Lambda' - \Lambda = -\Delta \leq \lambda_{\text{MAX}}$. Hence inequality (4.14) is true. From inequalities (4.13) and (4.14) it follows that

$$\int_{S \cap (0, \frac{\varepsilon}{\lambda_{\text{MAX}}})} \Lambda e^{-x\Lambda} dx + e^{-\Lambda \frac{\varepsilon}{\lambda_{\text{MAX}}}} \leq e^\varepsilon \left(\int_{S \cap (0, \frac{\varepsilon}{\lambda_{\text{MAX}}})} \Lambda' e^{-x\Lambda'} dx + e^{-\Lambda' \frac{\varepsilon}{\lambda_{\text{MAX}}}} \right). \quad (4.15)$$

Applying (4.10) and (4.11) to inequality (4.15), we obtain

$$P(\mathcal{A}_1(\mathfrak{M}_1) \in S) \leq e^\varepsilon P(\mathcal{A}_1(\mathfrak{M}_2) \in S).$$

4.4.2. Proof for multi-dimensional vectors

We have proved that DP-ExpSketch returning one-dimensional vectors is ε -differentially private. The version of the algorithm that returns a vector with m fields is a composition of m independent instances of the one-dimensional DP-ExpSketch, each of which is ε -differentially private. By Theorem 1.3, DP-ExpSketch returning a vector with m fields is $m\varepsilon$ -differentially private. $\square$

4.5. REDUCING NOISE WITH THE ADVANCED COMPOSITION THEOREM

By applying Theorem 1.4, we can replace the pure differentially private version of ExpSketch with an approximately differentially private one. The advantage of such a version is that, if a small risk of privacy leakage is accepted, the amount of noise added to the output can be reduced significantly. Theorem 1.4 implies that for ExpSketch with m fields to be (ε, δ) -differentially private, it suffices that each field is generated in an ε' -differentially private manner, where

$$\varepsilon' = \frac{\varepsilon}{2\sqrt{2m \ln(1/\delta)}}.$$

Without this theorem, each field would have to be generated in an $\frac{\varepsilon}{m}$ -differentially private manner, which would result in significantly more noise. For example, for a small value of δ , such as $\delta = 10^{-3}$, and ExpSketch with 10^6 fields, each field can be generated in an $\frac{\varepsilon}{7434}$ -differentially private manner instead of an $\frac{\varepsilon}{10^6}$ -differentially private one. This significantly reduces the noise and increases the accuracy of the answers while maintaining a high level of privacy, at the cost of accepting a small risk of privacy leakage.

4.6. THE UNMODIFIED EXPONENTIAL SKETCH IS APPROXIMATELY DIFFERENTIALLY PRIVATE FOR A CERTAIN CLASS OF DATA STREAMS

The aim of this section is to investigate whether the one-dimensional version of ExpSketch, without any modifications, satisfies approximate differential privacy (Definition 1.5). The following theorem shows that if the set of possible data streams is restricted in a certain way, then ExpSketch is approximately differentially private. The proof of Theorem 4.2 can be found in Appendix A.

Theorem 4.2 (Approximate differential privacy of ExpSketch). *Let $\varepsilon > 0$ and $\delta > 0$, and let us restrict the set of possible data streams to a subset such that for any two data streams $\mathfrak{M}$ and $\mathfrak{M}'$ from this subset that differ in one element, the following conditions are fulfilled:*

1. $\phi \in (0, e^{-\varepsilon}) \implies 1 + e^{\varepsilon} \left(e^{-\phi \frac{\ln(\phi)+\varepsilon}{\phi-1}} - 1 \right) - e^{-\frac{\ln(\phi)+\varepsilon}{\phi-1}} \leq \delta,$
2. $\phi \in (1, \infty) \implies e^{\frac{\ln(\phi)+\varepsilon}{1-\phi}} - e^{\varepsilon+\phi \frac{\ln(\phi)+\varepsilon}{1-\phi}} \leq \delta,$

where $\phi = \Lambda' / \Lambda$ is the ratio of the sums Λ' and Λ of the attributes of unique elements in $\mathfrak{M}'$ and $\mathfrak{M}$, respectively. Then the one-dimensional version of ExpSketch is (ε, δ) -differentially private on this subset of streams.

The next theorem gives an example of such a restriction on data streams. Its proof can be found in Appendix B.

Theorem 4.3 (Approximate differential privacy of ExpSketch for bounded attributes). *Consider the subset of data streams in which the sum of the attributes of unique elements of each stream is at least $\Lambda_{\min}$ and the attribute of each element is at most $\lambda_{\max} = \Lambda_{\min} (e^{\varepsilon} - 1)$. On this subset, the one-dimensional version of ExpSketch is (ε, δ) -differentially private for*

$$\delta = e^{\frac{2\varepsilon}{1-e^{\varepsilon}}} - e^{\varepsilon \frac{1+e^{\varepsilon}}{1-e^{\varepsilon}}}.$$

4.7. DIFFERENTIAL PRIVACY OF THE FAST EXPONENTIAL SKETCH

The article [11] introduces an optimization of ExpSketch called FastExpSketch. The algorithm still produces a vector of values drawn from the distribution $\text{Exp}(\Lambda)$. The optimization is based on the following theorem.

Theorem 4.4 ([11]). *Let $E_1, \dots, E_m$ be a sequence of i.i.d. exponential random variables and let*

$$E_{(1)} \leq E_{(2)} \leq \dots \leq E_{(m)}$$

denote their order statistics, i.e. $E_{(k)}$ denotes the k -th smallest value. Then for each $k \in \{1, 2, \dots, m\}$ we have the distributional equality

$$E_{(k)} \equiv E_{(k-1)} + \frac{E_k}{m - k + 1},$$

where $E_{(0)} = 0$.

Assume that we have generated variables $E_1, \dots, E_m$ from the distribution $\text{Exp}(\Lambda)$. After sorting them, we obtain $E_{(1)} \leq \dots \leq E_{(d)} \leq \dots \leq E_{(m)}$. If $E_{(d)} \geq \max\{\mathbf{M}_1, \dots, \mathbf{M}_m\}$, then the variables $E_{(d)}, \dots, E_{(m)}$ do not affect the vector $\mathbf{M}$. Thanks to the above theorem, FastExpSketch can directly generate the order statistics $E_{(1)}, \dots, E_{(d)}$, stopping as soon as $E_{(d)} \geq \max\{\mathbf{M}_1, \dots, \mathbf{M}_m\}$. The output of FastExpSketch is still a vector of values drawn from the distribution $\text{Exp}(\Lambda)$. Therefore, implementing differential privacy in FastExpSketch in the same way as in ExpSketch also yields a differentially private algorithm. FastExpSketch and DP-FastExpSketch are presented in Algorithms 5 and 6, respectively.

Algorithm 5 FastExpSketch($\mathfrak{M}$, m)

```
1: Initialization:
2:  $permInit \leftarrow (1, 2, \dots, m)$ 
3: Set each of the  $m$  positions of the data sketch  $\mathbf{M} = (\mathbf{M}_1, \dots, \mathbf{M}_m)$  to  $\infty$ 
4:  $MAX \leftarrow \infty$ 
5:
6: Update the data sketch  $\mathbf{M}$  with the elements of the data stream  $\mathfrak{M}$ :
7: for all  $(i, \lambda_i) \in \mathfrak{M}$  do
8:    $S \leftarrow 0$ 
9:    $updateMAX \leftarrow \text{false}$ 
10:   $P \leftarrow permInit$ 
11:  for  $k \leftarrow 1$  to  $m$  do
12:     $U_{i,k} \leftarrow \mathcal{H}(i \parallel k)$  ▷ hash  $\mathcal{H}(x) \in [0, 1)$ 
13:     $E_{i,k} \leftarrow -\frac{\ln(U_{i,k})}{\lambda_i}$ 
14:     $S \leftarrow S + \frac{E_{i,k}}{m-k+1}$ 
15:    if  $S \geq MAX$  then
16:      break
17:    end if
18:     $r \leftarrow \text{RandomInteger}([k, m], seed = i)$ 
19:     $\text{swap}(P[k], P[r])$ 
20:     $j \leftarrow P[k]$ 
21:    if  $\mathbf{M}_j = MAX$  then
22:       $updateMAX \leftarrow \text{true}$ 
23:    end if
24:     $\mathbf{M}_j \leftarrow \min\{\mathbf{M}_j, S\}$ 
25:  end for
26:  if  $updateMAX$  then
27:     $MAX \leftarrow \max\{\mathbf{M}_1, \dots, \mathbf{M}_m\}$ 
28:  end if
29: end for
30:
31: Estimate the sum of the attributes of unique elements in  $\mathfrak{M}$ :
32: return  $\frac{m-1}{\sum_{k=1}^m \mathbf{M}_k}$ 
```

Algorithm 6 DP-FastExpSketch($\mathfrak{M}, m, \lambda_{\text{MAX}}, \varepsilon$)

1: **Modify $\mathfrak{M}$ (a preliminary step ensuring differential privacy):**
2: $\mathfrak{M}' \leftarrow \left\{ \left(\text{unique identifier}, \frac{\lambda_{\text{MAX}}}{e^{\varepsilon}-1} \right) \right\} \cup \left\{ (i, \min(\lambda_i, \lambda_{\text{MAX}})) : (i, \lambda_i) \in \mathfrak{M} \right\}$
3:
4: **Build the data sketch from the modified data stream $\mathfrak{M}'$:**
5: $\mathbf{M} \leftarrow \text{FastExpSketch}(\mathfrak{M}', m)$
6:
7: **After the following modification, the privacy conditions are satisfied:**
8: **for all** $i \in \{1, 2, \dots, m\}$ **do**
9: $\mathbf{M}'_i \leftarrow \min \left(\mathbf{M}_i, \frac{\varepsilon}{\lambda_{\text{MAX}}} \right)$
10: **end for**
11:
12: **Estimate the sum of the attributes of unique elements in $\mathfrak{M}$:**
13: **return** $\frac{m-1}{\sum_{k=1}^m \mathbf{M}'_k} - \frac{\lambda_{\text{MAX}}}{e^{\varepsilon}-1}$

5. ATTACKS ON DATA PRIVACY

It might seem that recovering private data from analyses that aggregate the data of many individuals is impossible, or that the recovered data is too inaccurate to be of practical use to an adversary. Unfortunately, publishing such an analysis without proper data protection mechanisms (such as differential privacy) can lead to a leakage of personal data. Personal data can be vulnerable to various attacks, including reconstruction attacks and tracing attacks.

5.1. RECONSTRUCTION ATTACKS

A reconstruction attack is an attempt to reconstruct individuals' private data stored in a database by means of specially crafted queries. These queries are answered by the mechanism responsible for analyzing the aggregated data. If this mechanism was built without appropriate privacy protection, it may allow an adversary to reconstruct the original database from the answers. Such attacks can lead to a violation of the privacy of the individuals concerned and to the misuse of their data.

5.1.1. A basic example of a reconstruction attack

The article [10] presents a reconstruction attack on data that resembles a simplified fragment of a census of a small group of individuals. It describes an intuitive method of conducting a reconstruction attack on the data from Table 5.1. Based on this table, a set of constraints is formulated, whose solution can be found, for example, using a SAT solver.

5.1.2. Reconstruction attack on the 2010 US Census data

Reconstruction attacks can also be applied to analyses of aggregated data concerning much larger groups of individuals, even if these analyses use some basic privacy protection mechanisms. An example is the US Census, whose results were published until 2010 with some noise added to protect privacy. The articles [13] and [1] discuss a possible scenario of a reconstruction attack on the 2010 US Census, which contains the personal data of millions of Americans. In the 2020 US Census, differential privacy was already used to protect personal data.

Table 5.1: A fragment of the sample table of aggregated data from [10]. (D) denotes values suppressed for confidentiality.

Statistic	Group	Count	Median age	Mean age
1A	Total population	7	30	38
2A	Female	4	30	33.5
	Male	3	30	44
2B	Black or African American	4	51	48.5
	White	3	24	24
3A	Single adults	4	(D)	54
	Married adults		51	
4A	Black or African American female	3	36	36.7
	Black or African American male		(D)	
	White male		(D)	
	White female		(D)	

5.1.3. Reconstruction attack on the Diffix system

As a result of legal regulations such as the GDPR, there is a demand for privacy-focused software. Unfortunately, some of these programs use a heuristic approach to privacy, which may threaten the privacy of the individuals whose data they process. The article [2] describes an interesting reconstruction attack on the Diffix system. Diffix allows analysts to query a dataset using a subset of SQL. Upon receiving a query, Diffix analyzes it and returns an answer with added noise. The amount of noise depends on the size of the data subset used in the query and on the complexity of the query: each additional condition in the query increases the noise in the answer. The primary goal of the Diffix project was to develop a system that would:

- allow analysts to ask any number of statistical queries;
- achieve a very high accuracy of the answers;
- protect the privacy of individuals' data.

Unfortunately, as follows from the theorems presented in [5], [7] and [8], which are described in the next section, such a system cannot exist.

5.2. THEORETICAL FOUNDATIONS OF RECONSTRUCTION ATTACKS

The article [5] examines a reconstruction attack on a very basic database. The database is a sequence of n bits $d_1, d_2, \dots, d_n$, where $d_i \in \{0, 1\}$ is the value of a private attribute of the user u_i . The mechanism M answers queries $q \subseteq [n]$, where $[n] = \{1, \dots, n\}$, with $M(q) = \sum_{i \in q} d_i + \mathcal{E}$, where $\mathcal{E}$ is the noise added to the answer. The main result of this work is a theorem that can be stated in a simplified form as follows.

Theorem 5.1 ([5]). *If the magnitude of the noise is $\mathcal{E} = o(\sqrt{n})$, then there exists a polynomial-time algorithm that reconstructs the database correctly on all but $o(n)$ entries.*

The attack is presented in Algorithm 7.

Algorithm 7 Reconstruction attack against noise $\mathcal{E} = o(\sqrt{n})$

```

1: Query phase:
2: Let  $t = n(\log n)^2$ .
3: for all  $j \in \{1, 2, \dots, t\}$  do
4:   Choose uniformly at random  $q_j \subseteq [n]$ 
5:    $\tilde{a}_{q_j} \leftarrow M(q_j)$ 
6: end for
7:
8: Weeding phase:
9: Solve the following linear program with unknowns  $c_1, \dots, c_n$ :
   
$$\begin{aligned} \tilde{a}_{q_j} - \mathcal{E} &\leq \sum_{i \in q_j} c_i \leq \tilde{a}_{q_j} + \mathcal{E} && \text{for } 1 \leq j \leq t, \\ 0 &\leq c_i \leq 1 && \text{for } 1 \leq i \leq n. \end{aligned}$$

10:
11: Rounding phase:
12: Let  $c'_i = 1$  if  $c_i > 1/2$  and  $c'_i = 0$  otherwise. Output  $c'$ .

```

The above attack can be generalized to more complex databases. It is important to note that noise of magnitude $\mathcal{E} = \omega(\sqrt{n})$, for which the adversary cannot execute this attack successfully, does not by itself guarantee privacy: one can construct mechanisms in which the magnitude of the noise is $\mathcal{E} = \omega(\sqrt{n})$ and privacy is still compromised. The main conclusion of this study is that no mechanism can ensure privacy if it answers too many queries with too little noise. The articles [7] and [8] extend these results on reconstruction attacks.

5.3. A PRIVACY ATTACK ON EXPSKETCH

Let us consider a scenario with $n+1$ users. Each user U_i has made a transaction of value T_i for a product belonging to some subset of the set of categories $C = \{C_1, C_2, \dots, C_p\}$ (Tables 5.2 and 5.3).

Table 5.2: Example of assigning user transactions to categories.

	U_1	U_2	$\dots$	U_{n+1}
C_1	✓		$\dots$	✓
C_2		✓	$\dots$	
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
C_p			$\dots$	✓

Table 5.3: Example of assigning values to transactions.

	U_1	U_2	$\cdots$	U_{n+1}
T	\$100	\$50	$\cdots$	\$10

There are also p services, and each service S_i maintains its own data sketch, which processes all transactions belonging to the category C_i . Each service S_i can thus provide a data sketch describing the sum of all unique transactions belonging to the category C_i . By combining data sketches, it is possible to answer the question of what the sum of unique transactions belonging to any of several categories, for example C_1 , C_2 and C_{15} , is.

The administrator of these services would like to allow interested analysts to obtain answers to the question “what is the sum of unique transactions belonging to the given categories?”, but in such a way that the categories of individual transactions remain confidential.

In this scenario, let us assume that the adversary knows the content of Table 5.3 for all users and the content of Table 5.2 for all users except U_{n+1} . The adversary’s goal is to determine the categories of the transaction of the user U_{n+1} . To this end, the adversary uses their knowledge of the other users’ transactions and sends queries to the administrator.

To each query $Q_k \subseteq C$, the administrator responds with

$$A_k = \sum_{i=1}^{n+1} T_i \cdot g(T_i, Q_k),$$

where $g(T_i, Q_k)$ equals 1 if the transaction T_i has a category that belongs to Q_k , and 0 otherwise. Thanks to their knowledge, the adversary can compute the answers to their queries for all users except U_{n+1} on their own. This means that the adversary knows the values

$$A'_k = \sum_{i=1}^n T_i \cdot g(T_i, Q_k)$$

and can use them, together with the answers to their queries, to calculate $g(T_{n+1}, Q_k)$ for any query Q_k :

$$g(T_{n+1}, Q_k) = \frac{A_k - A'_k}{T_{n+1}}.$$

The adversary can use mixed-integer programming to formulate an optimization problem whose solution is a guess of the categories of the transaction T_{n+1} . The reconstruction algorithm assigns categories to the transaction T_{n+1} in such a way that the answers computed by the adversary, based on their knowledge of the other transactions and on the categories assigned to T_{n+1} , are as close as possible to the answers obtained from the administrator.

To protect the privacy of transaction data, the administrator should avoid giving exact answers. Instead, the answers should contain noise that reduces the effectiveness of the

adversary's reconstruction algorithm. To assess the effectiveness of the reconstruction attack, we use the F1-score, which measures how well the adversary has reconstructed the set of categories of the given transaction. The F1-score is the harmonic mean of precision and recall.

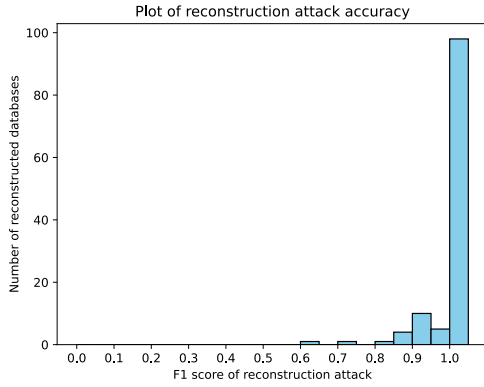
According to Theorems 4.2 and 4.3, if the value of a single transaction does not significantly affect the sum of all unique transactions in a given data sketch, approximate differential privacy is maintained and the reconstruction attack should be less effective. Below we present simulation results for two cases, depending on whether the transaction T_{n+1} has a significant impact on the data sketch.

5.3.1. Simulation of the attack when the transaction significantly affects the data sketch

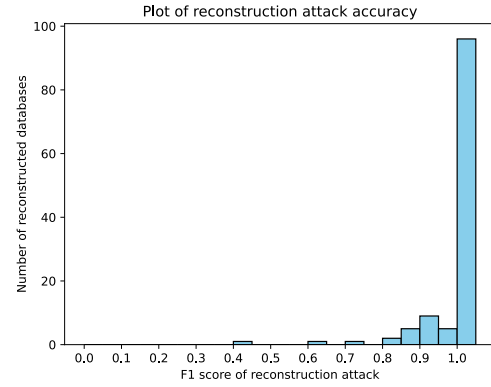
In this simulation, data streams consisting of 10 000 transactions were used. Each transaction could belong to any of the 20 available categories. The value of the transaction T_{n+1} accounted for over 10% of the total value of all transactions. For the differentially private algorithms, $\varepsilon = 1$ and $\delta = 10^{-4}$. The adversary could ask 100 random queries. The plots in Figure 5.1 show that in such extreme circumstances, when a single element has a significant impact on the data sketch, privacy may not be protected: the reconstruction attack, whether based on exact answers or on answers computed from the data sketch, has a high probability of success. The plots also show that both pure and approximate differential privacy protect against the reconstruction attack.

5.3.2. Simulation of the attack when the transaction does not significantly affect the data sketch

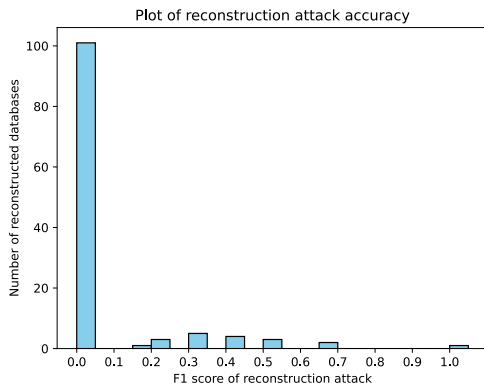
In this simulation, the value of the transaction T_{n+1} is reduced to the value of a regular transaction, so that its impact on the data sketch is the same as that of the other transactions. The plots in Figure 5.2 show that, based on exact answers, it is still possible to reconstruct the categories of the transaction T_{n+1} . However, the standard version of FastExpSketch already introduces enough noise to prevent the attack defined above.



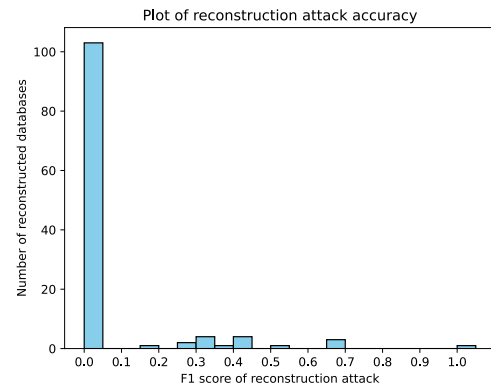
(a) Exact answers.



(b) FastExpSketch answers.

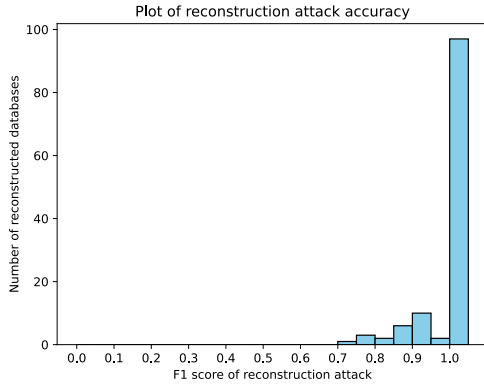


(c) (ϵ, δ) -DP FastExpSketch answers.

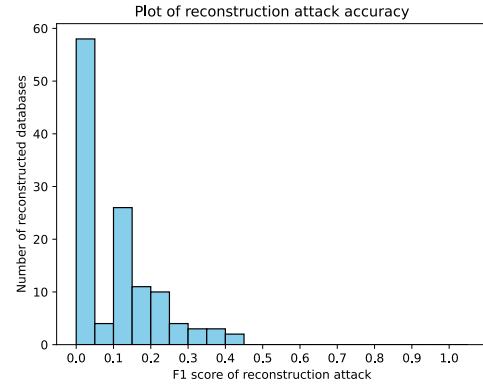


(d) ϵ -DP FastExpSketch answers.

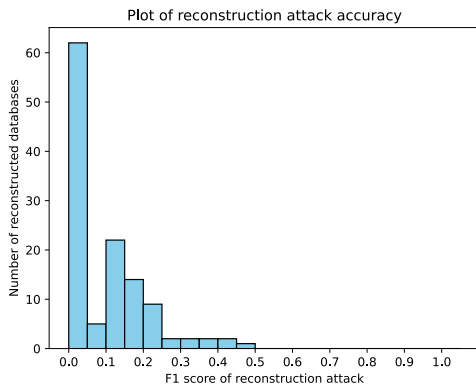
Fig. 5.1: Accuracy (F1-score) of the reconstruction attack on a *large* transaction, depending on the mechanism used to answer the queries.



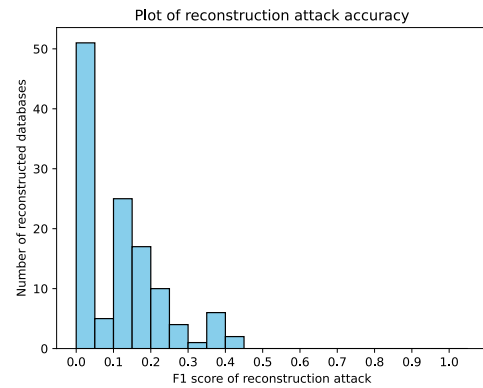
(a) Exact answers.



(b) FastExpSketch answers.



(c) (ϵ, δ) -DP FastExpSketch answers.



(d) ϵ -DP FastExpSketch answers.

Fig. 5.2: Accuracy (F1-score) of the reconstruction attack on a *regular* transaction, depending on the mechanism used to answer the queries.

6. EXPERIMENTAL ANALYSIS OF THE ADAPTED EXPONENTIAL SKETCH

Recall that the differentially private version of ExpSketch (Algorithm 4) starts by creating an artificial element with a very large attribute, which is then processed by ExpSketch. In the proof of Theorem 4.1, this element is used only to guarantee that the sum of the attributes of unique elements of the processed data stream exceeds a certain value. The attribute of the artificial element depends on the configuration of DP-ExpSketch, namely on the parameters ε , m and λ_{MAX} . If the configuration is such that this attribute is close to the sum of the attributes of unique elements of the data stream, the estimate will be very inaccurate. Conversely, a configuration in which this attribute is much smaller than the sum of the attributes of unique elements maintains a satisfactory estimation accuracy. Moreover, if it is known that DP-ExpSketch will only process data streams in which the sum of the attributes of unique elements exceeds this value, the artificial element can be omitted while still maintaining differential privacy. This results in a significantly higher accuracy of DP-ExpSketch.

6.1. THE ARTIFICIAL ELEMENT DOES NOT SIGNIFICANTLY AFFECT THE DATA STREAM

Figure 6.1 compares the accuracy of ExpSketch and its differentially private version. The algorithms were run on a stream of 100 000 elements, with an upper bound of 100 on the attribute of each element. The size m of the data sketch ranged from 2 to 100. The private version of the algorithm was $(0.1, 0)$ -differentially private.

By Theorem 4.1, for parameters m and ε , DP-ExpSketch is $(m\varepsilon, 0)$ -differentially private. Therefore, for a given m , the parameter ε was set to $\frac{0.1}{m}$. The plot shows that for the considered data, the accuracy of the private data sketch is very close to that of its non-private version. However, as the size of the data sketch increases, the attribute of the artificial element also increases, which results in a decrease in estimation accuracy.

6.2. THE ARTIFICIAL ELEMENT SIGNIFICANTLY AFFECTS THE DATA STREAM

In this experiment, a data stream configuration similar to that of the previous section was used, but the privacy parameter ε was decreased from 0.1 to 0.0001, which significantly

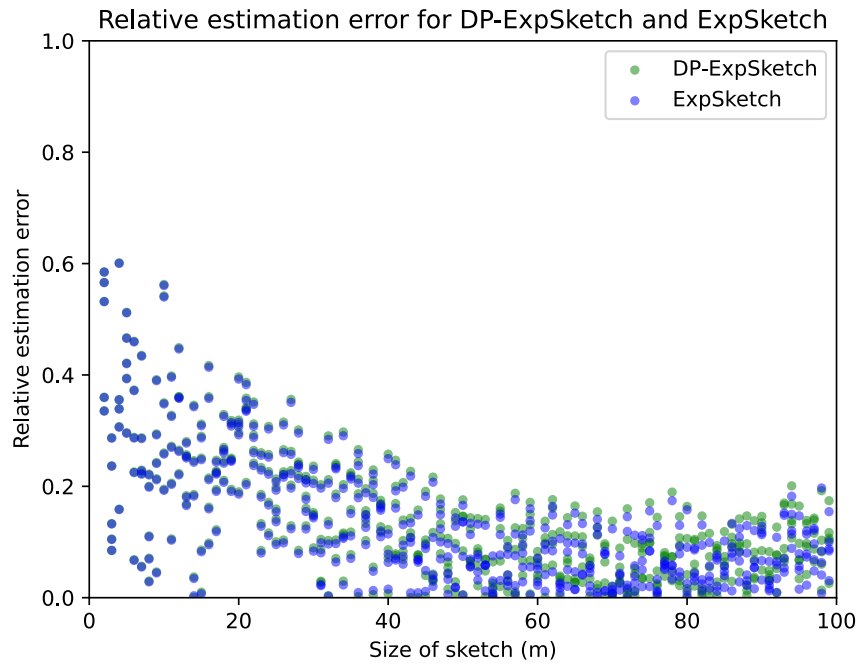


Fig. 6.1: Relative error of DP-ExpSketch and ExpSketch when the artificial element does not significantly affect the data stream.

increases the attribute of the artificial element. In this configuration, the artificial element has a substantial impact on the sum of the attributes of unique elements of the data stream, which results in inaccurate estimates of DP-ExpSketch (Figure 6.2).

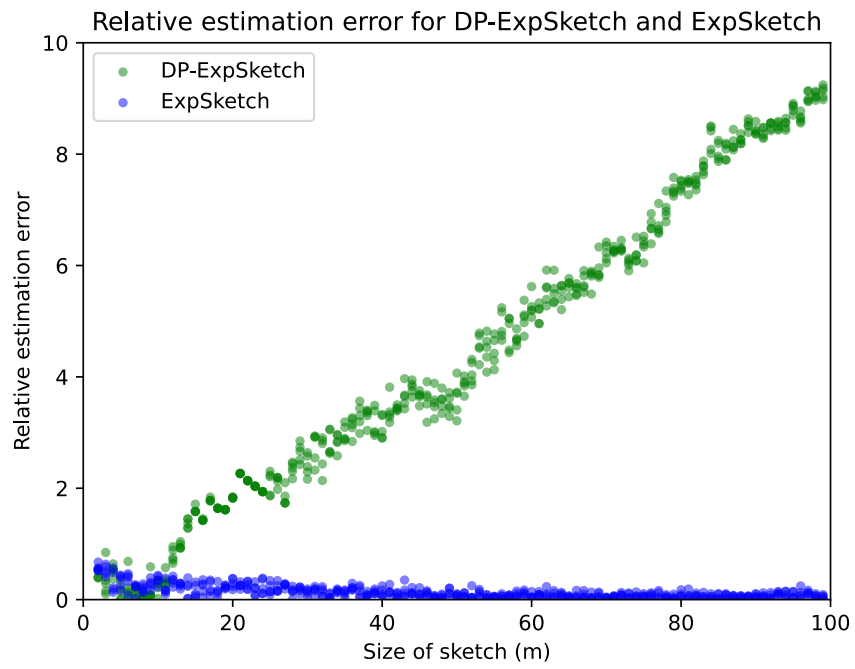


Fig. 6.2: Relative error of DP-ExpSketch and ExpSketch when the artificial element significantly affects the data stream.

SUMMARY

Data sketches can be vulnerable to privacy attacks, which makes it necessary to adapt these algorithms to the requirements of differential privacy. This thesis focuses on a theoretical and experimental analysis of data privacy in the ExpSketch algorithm and on the adaptation of this sketch to differential privacy. A simulation of a potential adversary demonstrated the effectiveness of the proposed differentially private adaptation of the data sketch. Furthermore, it was proven that if the set of possible data streams is restricted to a subset that excludes certain extreme cases, ExpSketch without any modifications satisfies approximate differential privacy.

BIBLIOGRAPHY

- [1] Abowd, J.M., Adams, T., Ashmead, R., Darais, D., Dey, S., Garfinkel, S.L., Goldschlag, N., Kifer, D., Leclerc, P., Lew, E., Moore, S., Rodríguez, R.A., Tadros, R.N., Vilhuber, L., *The 2010 census confidentiality protections failed, here's how and why*. 2022.
- [2] Cohen, A., Nissim, K., *Linear program reconstruction in practice*. 2020.
- [3] Desfontaines, D., Lochbihler, A., Basin, D., *Cardinality estimators do not preserve privacy*.
- [4] Dickens, C., Thaler, J., *Order-invariant cardinality estimators are differentially private*.
- [5] Dinur, I., Nissim, K., *Revealing information while preserving privacy*. 2003.
- [6] Dwork, C., Roth, A., *The algorithmic foundations of differential privacy*. 2014.
- [7] Dwork, C., McSherry, F., Talwar, K., *The price of privacy and the limits of LP decoding*. 2007.
- [8] Dwork, C., Yekhanin, S., *New efficient attacks on statistical disclosure control mechanisms*. 2008.
- [9] Flajolet, P., Fusy, É., Gandouet, O., Meunier, F., *HyperLogLog: the analysis of a near-optimal cardinality estimation algorithm*.
- [10] Garfinkel, S.L., Abowd, J.M., Martindale, C., *Understanding database reconstruction attacks on public data*. 2018.
- [11] Lemiesz, J., *Efficient framework for operating on data sketches*.
- [12] Lemiesz, J., *On the algebra of data sketches*.
- [13] Muralidhar, K., *A re-examination of the census bureau reconstruction and reidentification attack*. 2020.
- [14] Smith, A., Song, S., Thakurta, A., *The Flajolet-Martin sketch itself preserves differential privacy: Private counting with minimal space*.
- [15] Tschorsch, F., Scheuermann, B., *An algorithm for privacy-preserving distributed user statistics*.

LIST OF FIGURES

2.1	Data sketch usage diagram.	9
2.2	FM sketch.	11
2.3	Merge operation of two FM sketches.	11
2.4	Extended FM sketch.	12
2.5	Merge operation of two extended FM sketches.	12
3.1	Adversary model from [3].	15
3.2	Data sketches in a distributed system, as considered in [15].	16
3.3	The first access point aggregates data from the other access points.	17
5.1	Accuracy (F1-score) of the reconstruction attack on a <i>large</i> transaction, depending on the mechanism used to answer the queries.	38
5.2	Accuracy (F1-score) of the reconstruction attack on a <i>regular</i> transaction, depending on the mechanism used to answer the queries.	39
6.1	Relative error of DP-ExpSketch and ExpSketch when the artificial element does not significantly affect the data stream.	41
6.2	Relative error of DP-ExpSketch and ExpSketch when the artificial element significantly affects the data stream.	42

LIST OF ALGORITHMS

1	MinCount($\mathfrak{M}, k$)	10
2	DP-cardinality-estimator($\mathfrak{M}, \varepsilon$)	18
3	ExpSketch($\mathfrak{M}, m$)	19
4	DP-ExpSketch($\mathfrak{M}, m, \lambda_{\text{MAX}}, \varepsilon$)	24
5	FastExpSketch($\mathfrak{M}, m$)	31
6	DP-FastExpSketch($\mathfrak{M}, m, \lambda_{\text{MAX}}, \varepsilon$)	32
7	Reconstruction attack against noise $\mathcal{E} = o(\sqrt{n})$	35

LIST OF TABLES

5.1	A fragment of the sample table of aggregated data from [10]. (D) denotes values suppressed for confidentiality.	34
5.2	Example of assigning user transactions to categories.	35
5.3	Example of assigning values to transactions.	36
A.1	Properties of the function f depending on the value of ϕ	53

Appendix

A. PROOF OF THEOREM 4.2

Let us assume that S is a subset of $(0, \infty)$, and that Λ and Λ' are the sums of the attributes of unique elements in two data streams $\mathfrak{M}$ and $\mathfrak{M}'$, respectively. To show that ExpSketch without any modifications satisfies approximate differential privacy, we need to find a value of δ (preferably the smallest possible) for which the following condition is satisfied:

$$\Pr[\mathcal{M}(\mathfrak{M}) \in S] \leq e^\varepsilon \Pr[\mathcal{M}(\mathfrak{M}') \in S] + \delta. \quad (\text{A.1})$$

Condition (A.1) can be written as

$$\int_S \Lambda e^{-x\Lambda} dx \leq e^\varepsilon \int_S \Lambda' e^{-x\Lambda'} dx + \delta. \quad (\text{A.2})$$

Let us assume that $\Lambda \neq \Lambda'$, because if $\Lambda = \Lambda'$, condition (A.1) is trivially fulfilled. This assumption will be needed in the subsequent computations. Let us define

$$\phi = \frac{\Lambda'}{\Lambda}.$$

Then $\phi \in (0, 1) \cup (1, \infty)$. Since $\Lambda' = \Lambda\phi$, inequality (A.2) can be written as

$$\int_S \Lambda e^{-x\Lambda} dx \leq e^\varepsilon \int_S \Lambda\phi e^{-x\Lambda\phi} dx + \delta,$$

which is equivalent to

$$\int_S \left(\Lambda e^{-x\Lambda} - e^\varepsilon \Lambda\phi e^{-x\Lambda\phi} \right) dx \leq \delta. \quad (\text{A.3})$$

Let us define the function $f : (0, \infty) \rightarrow \mathbb{R}$:

$$f(x) = \Lambda e^{-x\Lambda} - e^\varepsilon \Lambda\phi e^{-x\Lambda\phi}.$$

Using f , inequality (A.3) can be written as

$$\int_S f(x) dx \leq \delta. \quad (\text{A.4})$$

In order to find the optimal δ for which inequality (A.4) is satisfied, we analyze the properties of the function f .

1. The derivative of the function:

$$\frac{df(x)}{dx} = e^\varepsilon \Lambda^2 \phi^2 e^{-\Lambda \phi x} - \Lambda^2 e^{-\Lambda x}.$$

2. The zero x_e of the derivative:

$$\begin{aligned} \frac{df(x_e)}{dx} &= 0, \\ e^\varepsilon \Lambda^2 \phi^2 e^{-\Lambda \phi x_e} &= \Lambda^2 e^{-\Lambda x_e}, \\ e^\varepsilon \phi^2 &= e^{-\Lambda x_e(1-\phi)}, \\ e^{2\ln(\phi)+\varepsilon} &= e^{-\Lambda x_e(1-\phi)}. \end{aligned}$$

Since the exponential function is injective,

$$\begin{aligned} 2\ln(\phi) + \varepsilon &= \Lambda x_e(\phi - 1), \\ x_e &= \frac{2\ln(\phi) + \varepsilon}{\Lambda(\phi - 1)}. \end{aligned}$$

3. The sign of x_e depends on the value of ϕ . Let us consider two cases, depending on whether ϕ is greater than 1.

(a) If $\phi > 1$, then x_e is positive.

(b) If $\phi < 1$, i.e. $\phi \in (0, 1)$, let us check for which values of ϕ the zero of the derivative is positive:

$$x_e = \frac{2\ln(\phi) + \varepsilon}{\Lambda(\phi - 1)} \geq 0. \quad (\text{A.5})$$

Since the denominator is negative, multiplying inequality (A.5) by $\Lambda(\phi - 1)$ reverses the inequality sign:

$$\begin{aligned} 2\ln(\phi) + \varepsilon &\leq 0, \\ \ln(\phi) &\leq -\frac{\varepsilon}{2}. \end{aligned}$$

Since the exponential function is increasing,

$$\phi \leq e^{-\frac{\varepsilon}{2}}.$$

To sum up:

$$\begin{aligned} \phi \in (0, e^{-\frac{\varepsilon}{2}}) \cup (1, \infty) &\implies x_e \geq 0, \\ \phi \in (e^{-\frac{\varepsilon}{2}}, 1) &\implies x_e \leq 0. \end{aligned}$$

4. The second derivative of the function:

$$\frac{d^2 f(x)}{dx^2} = \Lambda^3 e^{-\Lambda x} - e^\varepsilon \Lambda^3 \phi^3 e^{-\Lambda \phi x}.$$

5. The sign of the second derivative at x_e :

$$\begin{aligned} \frac{d^2 f(x_e)}{dx^2} &\geq 0, \\ \Lambda^3 e^{-\Lambda \frac{2 \ln(\phi) + \varepsilon}{\Lambda(\phi-1)}} - e^\varepsilon \Lambda^3 \phi^3 e^{-\Lambda \phi \frac{2 \ln(\phi) + \varepsilon}{\Lambda(\phi-1)}} &\geq 0, \\ e^{\frac{2 \ln(\phi) + \varepsilon}{1-\phi}} &\geq e^\varepsilon \phi^3 e^{\phi \frac{2 \ln(\phi) + \varepsilon}{1-\phi}}, \\ \frac{e^{\frac{2 \ln(\phi) + \varepsilon}{1-\phi}}}{e^{\phi \frac{2 \ln(\phi) + \varepsilon}{1-\phi}}} &\geq e^\varepsilon \phi^3, \\ e^{\frac{2 \ln(\phi) + \varepsilon}{1-\phi} - \phi \frac{2 \ln(\phi) + \varepsilon}{1-\phi}} &\geq e^\varepsilon \phi^3, \\ e^{\frac{(2 \ln(\phi) + \varepsilon)(1-\phi)}{1-\phi}} &\geq e^\varepsilon e^{3 \ln(\phi)}, \\ e^{2 \ln(\phi) + \varepsilon} &\geq e^\varepsilon e^{3 \ln(\phi)}, \\ \phi^2 &\geq \phi^3, \\ \phi &\leq 1. \end{aligned}$$

To sum up:

$$\phi \in (0, 1) \implies x_e \text{ is a minimum,}$$

$$\phi \in (1, \infty) \implies x_e \text{ is a maximum.}$$

6. The sign of the extremum:

$$\begin{aligned}
 f(x_e) &= \Lambda e^{\frac{2 \ln(\phi) + \varepsilon}{1-\phi}} - e^\varepsilon \Lambda \phi e^{\phi \frac{2 \ln(\phi) + \varepsilon}{1-\phi}} \geq 0, \\
 \Lambda e^{\frac{2 \ln(\phi) + \varepsilon}{1-\phi}} &\geq e^\varepsilon \Lambda \phi e^{\phi \frac{2 \ln(\phi) + \varepsilon}{1-\phi}}, \\
 \frac{e^{\frac{2 \ln(\phi) + \varepsilon}{1-\phi}}}{e^\varepsilon \phi e^{\phi \frac{2 \ln(\phi) + \varepsilon}{1-\phi}}} &\geq 1, \\
 \frac{e^{\frac{2 \ln(\phi) + \varepsilon}{1-\phi} - \phi \frac{2 \ln(\phi) + \varepsilon}{1-\phi}}}{e^\varepsilon \phi} &\geq 1, \\
 \frac{e^{\frac{(2 \ln(\phi) + \varepsilon)(1-\phi)}{1-\phi}}}{e^\varepsilon \phi} &\geq 1, \\
 \frac{e^{2 \ln(\phi) + \varepsilon}}{e^\varepsilon \phi} &\geq 1, \\
 \frac{\phi^2 e^\varepsilon}{e^\varepsilon \phi} &\geq 1, \\
 \phi &\geq 1.
 \end{aligned}$$

To sum up:

$$\phi \in (0, 1) \implies f(x_e) \leq 0,$$

$$\phi \in (1, \infty) \implies f(x_e) \geq 0.$$

7. The values of the function at the endpoints of the domain:

(a) Left endpoint:

$$f(0) = \Lambda(1 - e^\varepsilon \phi).$$

The sign of $f(0)$ depends on the value of ϕ :

$$f(0) = \Lambda(1 - e^\varepsilon \phi) \geq 0,$$

$$1 - e^\varepsilon \phi \geq 0,$$

$$\phi \leq e^{-\varepsilon}.$$

To sum up:

$$\phi \in (e^{-\varepsilon}, 1) \cup (1, \infty) \implies f(0) \leq 0,$$

$$\phi \in (0, e^{-\varepsilon}) \implies f(0) \geq 0.$$

(b) Right endpoint:

$$\lim_{x \rightarrow \infty} f(x) = 0.$$

8. The zero x_0 of the function f :

$$\begin{aligned}
 f(x_0) &= \Lambda e^{-x_0 \Lambda} - e^\varepsilon \Lambda \phi e^{-x_0 \Lambda \phi} = 0, \\
 \frac{e^{-x_0 \Lambda}}{e^{-x_0 \Lambda \phi}} &= e^{\ln(\phi) + \varepsilon}, \\
 e^{-x_0 \Lambda(1-\phi)} &= e^{\ln(\phi) + \varepsilon}, \\
 e^{x_0 \Lambda(\phi-1)} &= e^{\ln(\phi) + \varepsilon}.
 \end{aligned}$$

Since the exponential function is injective,

$$\begin{aligned}
 x_0 \Lambda(\phi - 1) &= \ln(\phi) + \varepsilon, \\
 x_0 &= \frac{\ln(\phi) + \varepsilon}{\Lambda(\phi - 1)}.
 \end{aligned}$$

9. The sign of the zero of f depends on the value of ϕ . Let us consider two cases, depending on whether ϕ is greater than 1.

(a) If $\phi > 1$, then the zero of f is positive.

(b) If $\phi < 1$, i.e. $\phi \in (0, 1)$, then

$$\frac{\ln(\phi) + \varepsilon}{\Lambda(\phi - 1)} \geq 0. \tag{A.6}$$

Since the denominator is negative, multiplying inequality (A.6) by $\Lambda(\phi - 1)$ reverses the inequality sign:

$$\begin{aligned}
 \ln(\phi) + \varepsilon &\leq 0, \\
 \phi &\leq e^{-\varepsilon}.
 \end{aligned}$$

To sum up:

$$\begin{aligned}
 \phi \in (e^{-\varepsilon}, 1) &\implies x_0 \leq 0, \\
 \phi \in (0, e^{-\varepsilon}) \cup (1, \infty) &\implies x_0 \geq 0.
 \end{aligned}$$

All the above properties of the function f are summarized in Table A.1.

Table A.1: Properties of the function f depending on the value of ϕ .

	x_0	x_e	$f(x_e)$	type of extremum	$f(0)$	$\lim_{x \rightarrow \infty} f(x)$
$\phi \in (0, e^{-\varepsilon})$	+	+	−	minimum	+	0
$\phi \in [e^{-\varepsilon}, e^{-\frac{\varepsilon}{2}})$	−	+	−	minimum	−	0
$\phi \in [e^{-\frac{\varepsilon}{2}}, 1)$	−	−	−	minimum	−	0
$\phi \in (1, \infty)$	+	+	+	maximum	−	0

From this table, it follows that:

$$\begin{aligned}
\phi \in (0, e^{-\varepsilon}) &\implies \forall S \subseteq (0, \infty) : \int_S f(x) \, dx \leq \int_0^{x_0} f(x) \, dx, \\
\phi \in [e^{-\varepsilon}, e^{-\frac{\varepsilon}{2}}) &\implies \forall S \subseteq (0, \infty) : \int_S f(x) \, dx \leq 0, \\
\phi \in [e^{-\frac{\varepsilon}{2}}, 1) &\implies \forall S \subseteq (0, \infty) : \int_S f(x) \, dx \leq 0, \\
\phi \in (1, \infty) &\implies \forall S \subseteq (0, \infty) : \int_S f(x) \, dx \leq \int_{x_0}^{\infty} f(x) \, dx.
\end{aligned}$$

Hence, it suffices to find a value of δ that bounds both the integral $\int_0^{x_0} f(x) \, dx$ for $\phi \in (0, e^{-\varepsilon})$ and the integral $\int_{x_0}^{\infty} f(x) \, dx$ for $\phi \in (1, \infty)$.

1. Let us consider the first case, $\phi \in (0, e^{-\varepsilon})$:

$$\begin{aligned}
\int_S f(x) \, dx &\leq \int_0^{x_0} f(x) \, dx = \int_0^{\frac{\ln(\phi)+\varepsilon}{\Lambda(\phi-1)}} \left(\Lambda e^{-x\Lambda} - e^{\varepsilon} \Lambda \phi e^{-x\Lambda\phi} \right) \, dx \\
&= \left[-e^{-\Lambda x} + e^{\varepsilon-\Lambda\phi x} \right]_0^{\frac{\ln(\phi)+\varepsilon}{\Lambda(\phi-1)}} \\
&= 1 + e^{\varepsilon} \left(e^{-\phi \frac{\ln(\phi)+\varepsilon}{\phi-1}} - 1 \right) - e^{-\frac{\ln(\phi)+\varepsilon}{\phi-1}}.
\end{aligned}$$

2. Let us now consider the second case, $\phi \in (1, \infty)$:

$$\begin{aligned}
\int_S f(x) \, dx &\leq \int_{x_0}^{\infty} f(x) \, dx = \int_{\frac{\ln(\phi)+\varepsilon}{\Lambda(\phi-1)}}^{\infty} \left(\Lambda e^{-x\Lambda} - e^{\varepsilon} \Lambda \phi e^{-x\Lambda\phi} \right) \, dx \\
&= \lim_{b \rightarrow \infty} \left[-e^{-\Lambda x} + e^{\varepsilon-\Lambda\phi x} \right]_{\frac{\ln(\phi)+\varepsilon}{\Lambda(\phi-1)}}^b \\
&= e^{-\Lambda \frac{\ln(\phi)+\varepsilon}{\Lambda(\phi-1)}} - e^{\varepsilon-\Lambda\phi \frac{\ln(\phi)+\varepsilon}{\Lambda(\phi-1)}} \\
&= e^{\frac{\ln(\phi)+\varepsilon}{1-\phi}} - e^{\varepsilon+\phi \frac{\ln(\phi)+\varepsilon}{1-\phi}}.
\end{aligned}$$

These are exactly the bounds in the conditions of Theorem 4.2, which completes the proof. $\square$

B. PROOF OF THEOREM 4.3

Let us assume that the set of possible data streams has been restricted to a subset in which the sum of the attributes of unique elements of each data stream is at least $\Lambda_{\min}$ and each element of the data streams has an attribute not greater than $\lambda_{\max} = \Lambda_{\min}(e^\varepsilon - 1)$.

First, let us prove that ϕ does not belong to the interval $(0, e^{-\varepsilon})$. Let us take two streams $\mathfrak{M}$ and $\mathfrak{M}'$ that differ in only one element, with sums of the attributes of unique elements equal to Λ and Λ' , respectively, where $\Lambda, \Lambda' \geq \Lambda_{\min}$ and each element of these data streams is not greater than $\lambda_{\max} = \Lambda_{\min}(e^\varepsilon - 1)$. Let us consider two cases, depending on whether Λ is greater than Λ' .

1. If $\Lambda' > \Lambda$, then $\phi = \frac{\Lambda'}{\Lambda} > 1$, so $\phi \notin (0, e^{-\varepsilon})$.
2. If $\Lambda' < \Lambda$, let us assume that the streams differ in the i -th element, and denote the i -th element of the first stream by λ_i and the i -th element of the second stream by λ'_i . Since $\Lambda > \Lambda'$, we have $\lambda_i > \lambda'_i$. Let $\Delta\lambda_i = \lambda_i - \lambda'_i$. It follows from the assumptions that

$$\Lambda_{\min}(e^\varepsilon - 1) = \lambda_{\max} \geq \Delta\lambda_i,$$

which is equivalent to

$$\begin{aligned} \frac{\Lambda_{\min}(1 - e^{-\varepsilon})}{e^{-\varepsilon}} &\geq \Delta\lambda_i, \\ \Lambda_{\min}(1 - e^{-\varepsilon}) &\geq e^{-\varepsilon}\Delta\lambda_i, \\ \Lambda_{\min} &\geq e^{-\varepsilon}\Delta\lambda_i + e^{-\varepsilon}\Lambda_{\min}, \\ \Lambda_{\min} &\geq e^{-\varepsilon}(\Lambda_{\min} + \Delta\lambda_i), \\ \frac{\Lambda_{\min}}{\Lambda_{\min} + \Delta\lambda_i} &\geq e^{-\varepsilon}. \end{aligned}$$

Since $\Lambda' \geq \Lambda_{\min}$ and the function $t \mapsto \frac{t}{t + \Delta\lambda_i}$ is increasing,

$$\phi = \frac{\Lambda'}{\Lambda} = \frac{\Lambda'}{\Lambda' + \Delta\lambda_i} \geq \frac{\Lambda_{\min}}{\Lambda_{\min} + \Delta\lambda_i} \geq e^{-\varepsilon}.$$

Hence $\phi \notin (0, e^{-\varepsilon})$, so the first condition of Theorem 4.2 is satisfied.

Now we can determine the largest possible value of δ in the case $\phi \in (1, \infty)$ for such a restricted set of data streams. For $\phi > 1$, the function

$$g(\phi) = e^{\frac{\ln(\phi) + \varepsilon}{1 - \phi}} - e^{\varepsilon + \phi \frac{\ln(\phi) + \varepsilon}{1 - \phi}}$$

is increasing. Therefore, an upper bound on this function over the restricted subset of data streams is $g(\phi_{\max})$, where $\phi_{\max}$ is the largest value of ϕ that can be obtained in this subset. It can be easily shown that the largest possible value of ϕ is attained when $\Lambda = \Lambda_{\min}$ and $\Lambda' = \Lambda + \lambda_{\max}$. In that case,

$$\phi_{\max} = \frac{\Lambda'}{\Lambda} = \frac{\Lambda + \lambda_{\max}}{\Lambda} = \frac{\Lambda_{\min} + \Lambda_{\min}(e^{\varepsilon} - 1)}{\Lambda_{\min}} = e^{\varepsilon}.$$

Hence, in this case we can take

$$\delta = g(\phi_{\max}) = g(e^{\varepsilon}) = e^{\frac{\ln(e^{\varepsilon}) + \varepsilon}{1 - e^{\varepsilon}}} - e^{\varepsilon + e^{\varepsilon} \frac{\ln(e^{\varepsilon}) + \varepsilon}{1 - e^{\varepsilon}}} = e^{\frac{2\varepsilon}{1 - e^{\varepsilon}}} - e^{\varepsilon \frac{1 + e^{\varepsilon}}{1 - e^{\varepsilon}}}. \quad \square$$